

计算机组成原理

第八章 CPU 的结构和功能



系统结构研究所

合肥工业大学计算机学院贾伟

大　纲

（一） CPU的功能和基本结构

（二） 指令执行过程

（三） 数据通路的功能和基本结构

（四） 控制器的功能和工作原理

1. 硬布线控制器

2. 微程序控制器

微程序、微指令和微命令；微指令格式；微命令的编码方式；微地址的形成方式

（五） 指令流水线

1. 指令流水线的基本概念

2 .指令流水线的基本实现

3. 超标量和动态流水线的基本概念

合肥工业大学计算机学院贾伟

Contents

8.1 CPU 的结构

8.2 指令周期

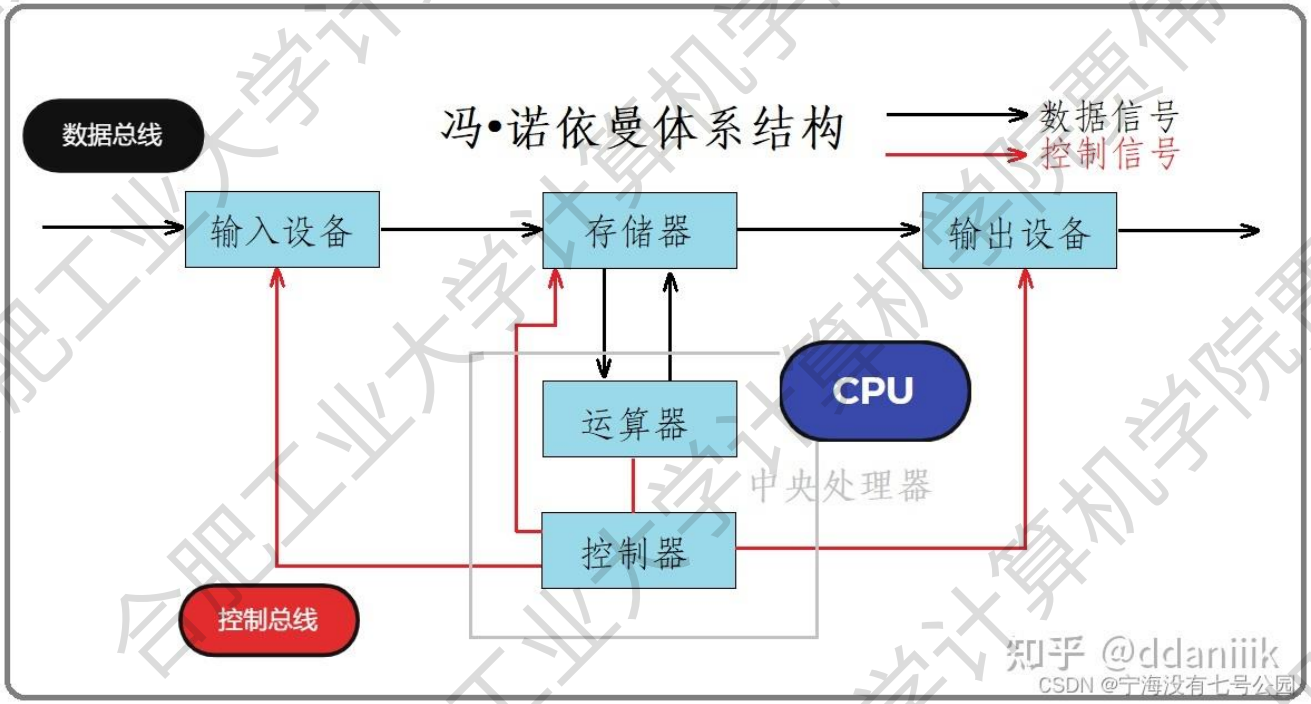
8.3 指令流水

8.4 中断系统

合肥工业大学计算机学院贾伟

一、 CPU 的功能

1. CPU实质包括运算器和控制器两大部分



冯·诺依曼体系结构

数学家冯·诺依曼提出了计算机制造的三个基本原则，即采用二进制逻辑、程序存储执行以及计算机由五个部分组成（运算器、控制器、存储器、输入设备、输出设备）

一、 CPU 的功能

1. 控制器的功能

- CPU是由**运算器**和**控制器**组成的
- 对于冯·诺依曼结构的计算机而言，一旦程序进入存储器后，就可由计算机自动完成取指令和执行指令的任务，**控制器**就是专用于完成此项任务的，它负责协调并控制计算机各部件执行程序的指令序列，其基本功能是取指令、分析指令和执行指令

合肥工业大学计算机学院贾伟

一、 CPU 的功能

1. 控制器的功能

(1) 取指令：

控制器自动形成指令地址，发出取指令的命令，将对应此地址的命令取到控制器中

(2) 分析指令：

① 分析指令要完成什么操作；② 分析操作数的有效地址

(3) 执行指令：

根据分析指令产生的“操作命令”和“操作数地址”，形成操作控制信号序列，通过对运算器、存储器以及I/O设备的操作，执行每条命令。

(4) 控制程序的输入和运算结果的输出

即控制主机与I/O设备交换信息

(5) 总线管理

(6) 处理中断的能力

处理机器运行过程中出现的异常情况(如掉电)和特殊请求(如打印机请求打印一行字符)

一、 CPU 的功能

【控制器实现】

- CPU必须具有控制程序的顺序执行(称**指令控制**)
- 产生完成每条指令所需的控制命令(称**操作控制**)
- 对各种操作加以时间上的控制(称**时间控制**)
- **处理中断**等功能。

【运算器实现】

- 对数据进行算数运算和逻辑运算(**数据加工**)

指令控制

操作控制

时间控制

处理中断

数据加工

CSDN @Aelief

8.1 CPU 的结构

一、 CPU 的功能

1. 控制器的功能

取指令

分析指令

执行指令，发出各种操作命令

控制程序输入及结果的输出

总线管理

处理异常情况和特殊请求

指令控制

操作控制

时间控制

处理中断

2. 运算器的功能

实现算术运算和逻辑运算

数据加工

二、CPU 结构框图

1. CPU 与系统总线　　根据CPU的功能不难设想：

(1) 要取指令，必须有一个**寄存器**专用于存放当前指令的地址

(2) 要分析指令，必须有存放当前指令的**寄存器**和对指令操作码进行译码的部件(指令译码器ID)

(3) 要执行指令，必须有一个能发出各种操作命令序列的控制部件**CU**

(4) 要完成算术运算和逻辑运算，必须有存放操作数的寄存器和实现算逻运算的部件**ALU**

(5) 为了处理异常情况和特殊请求，还必须有**中断系统**

可见，CPU可由四大部分组成

二、CPU 结构框图

1. CPU 与系统总线

(1) **指令控制**：由程序计数器PC(表示指令的在主存中的地址)和指令寄存器IR(存放指令)完成

(2) **操作控制**和**时间控制**：由控制部件CU和时序电路完成

(3) **数据加工**：由ALU和寄存器完成

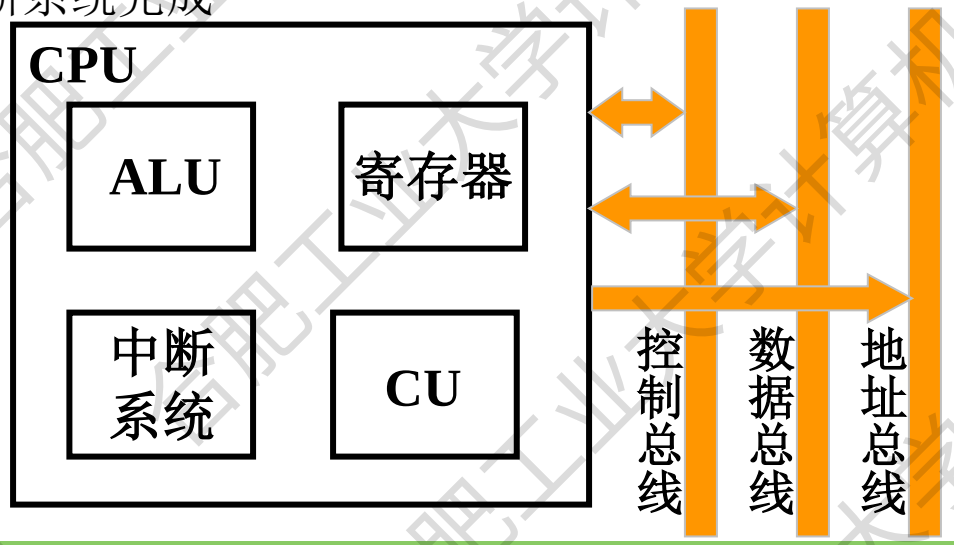
(4) **处理中断**：由中断系统完成

指令控制 → PC IR

操作控制、时间控制 → CU 时序电路

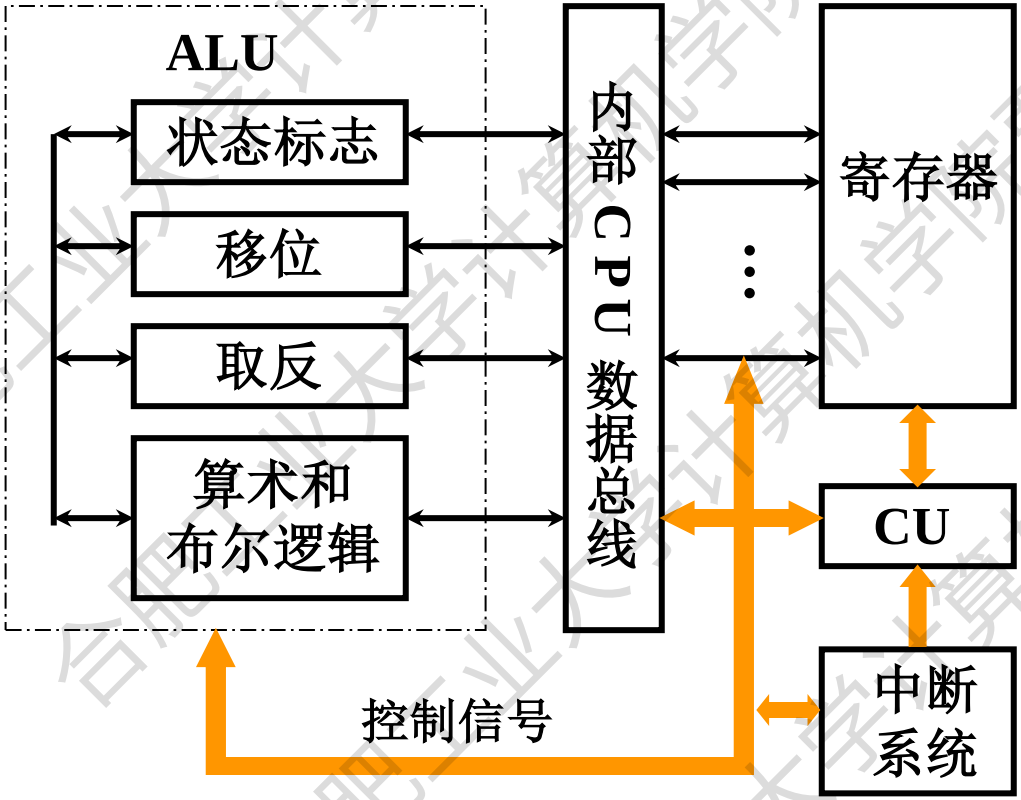
数据加工 → ALU 寄存器

处理中断 → 中断系统



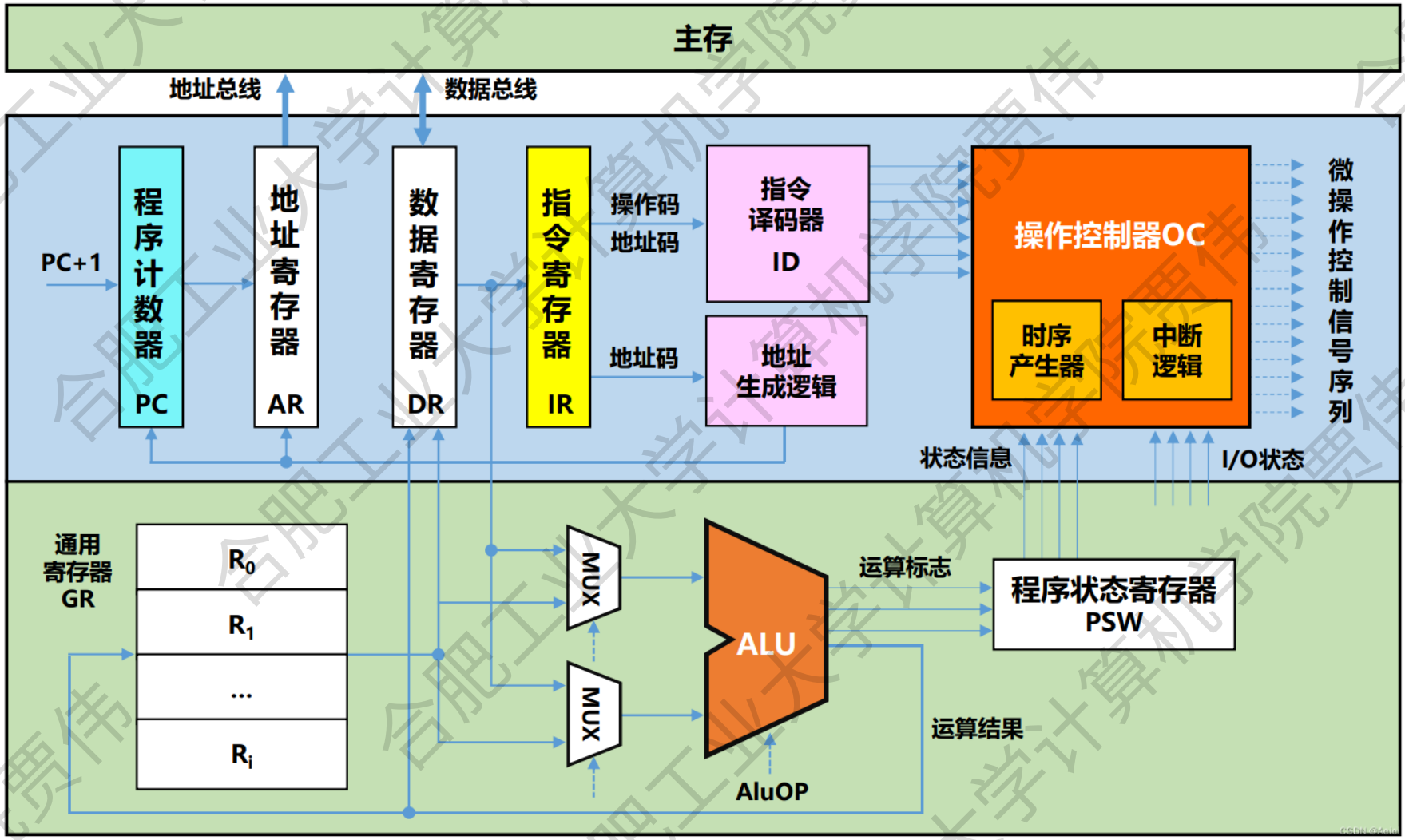
2. CPU 的内部结构

将上图细化，又可得下图。图中ALU部件实际上只对CPU内部寄存器的数据进行操作。



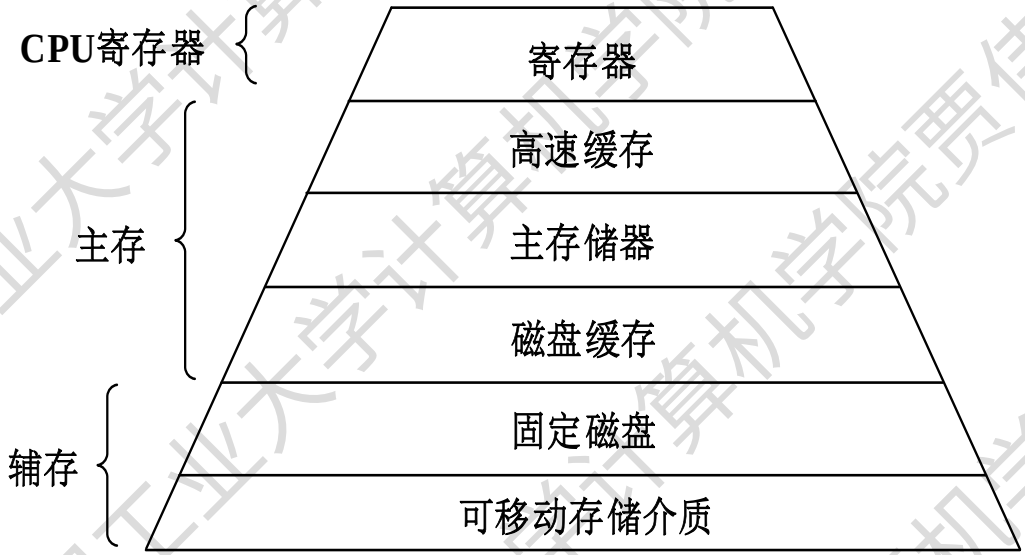
2. CPU 的内部结构

一个简单的CPU模型



三、 CPU 的寄存器

多层结构的存储系统



高速缓存：

Data Cache

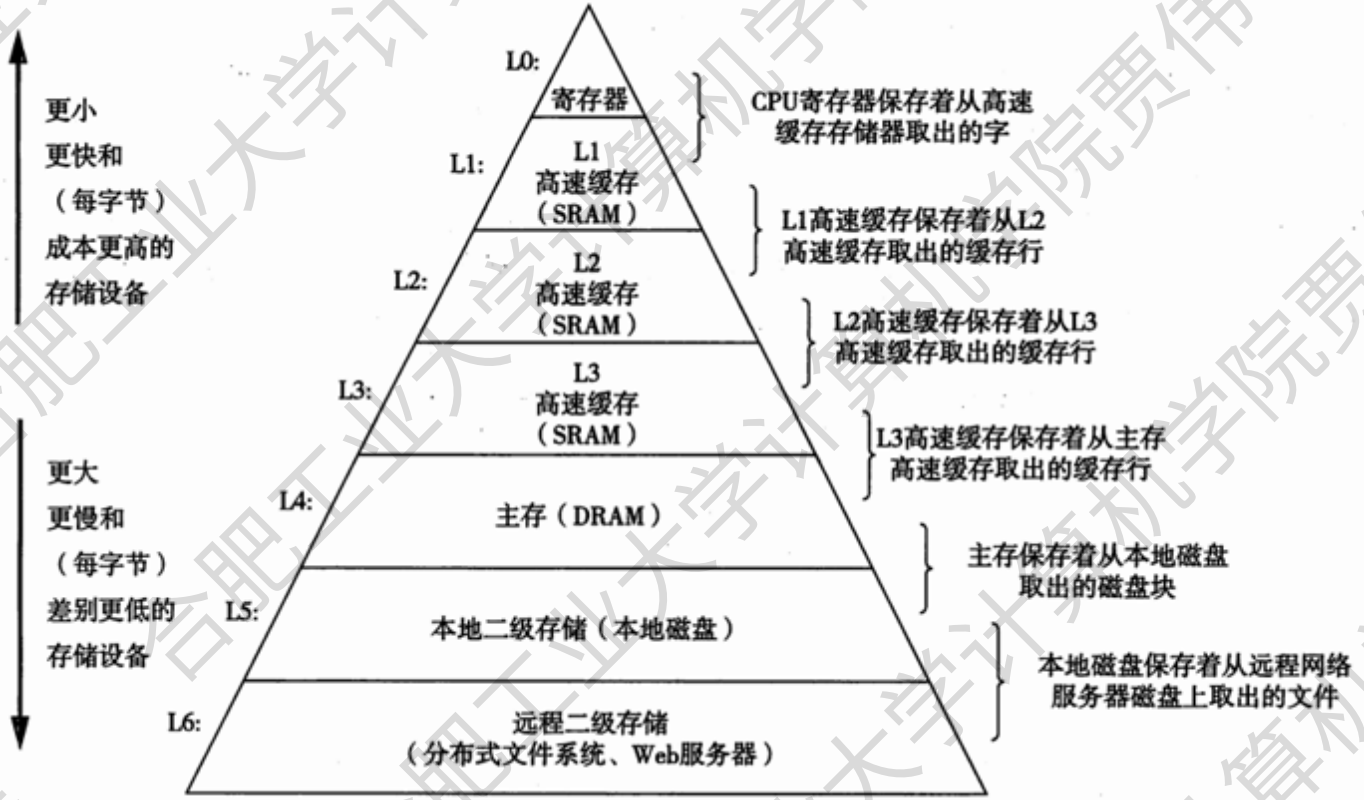
TLB(Translation Lookaside Buffer)

内存：DRAM, SDRAM等；

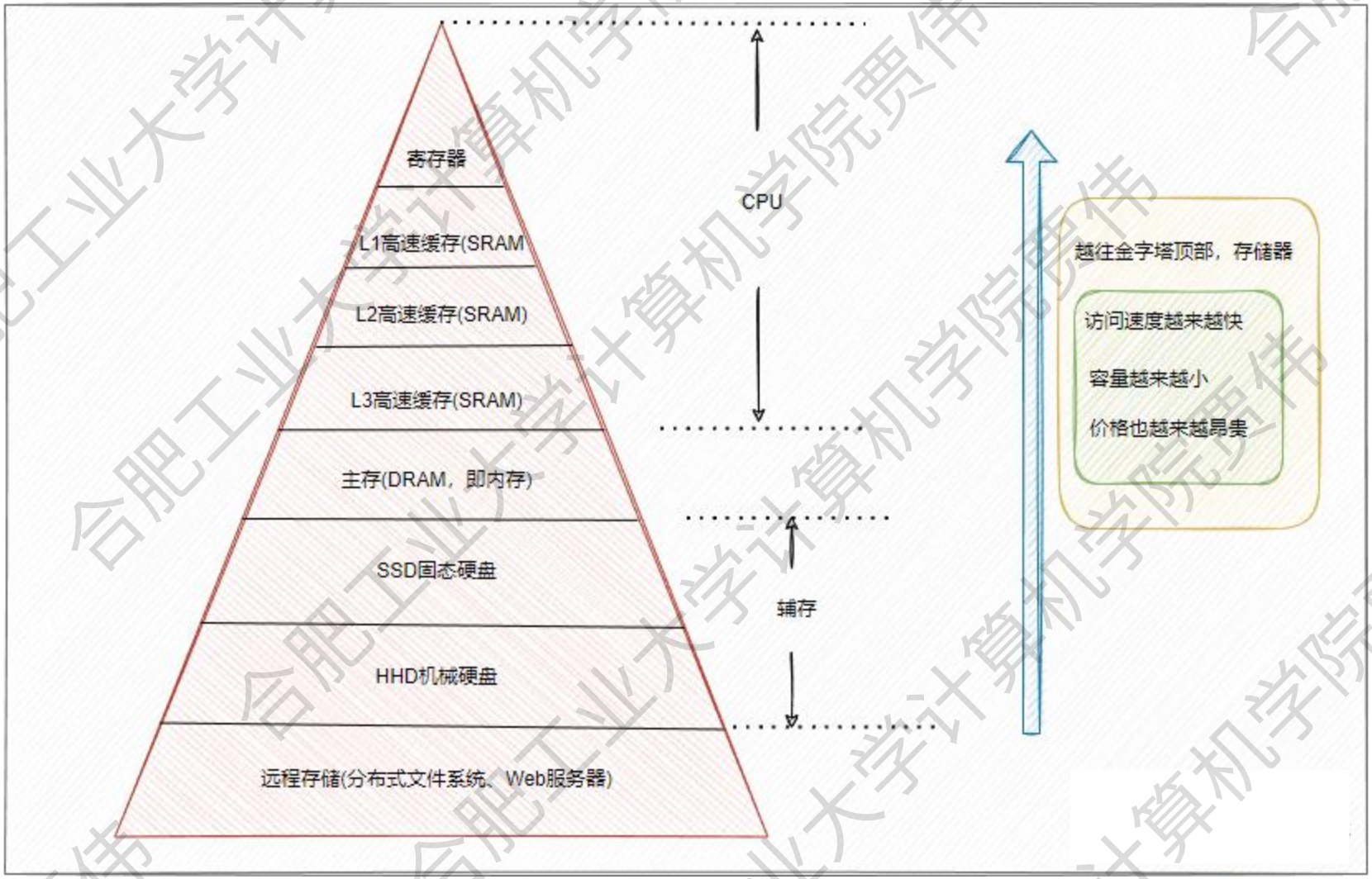
外存：软盘、硬盘、光盘、磁带、移动硬盘、U盘、SD卡等；

三、 CPU 的寄存器

Intel i7的存储器层次结构



三、 CPU 的寄存器


寄存器
L1高速缓存(SRAM
L2高速缓存(SRAM)
L3高速缓存(SRAM)
主存(DRAM，即内存)
SSD固态硬盘
HHD机械硬盘
远程存储(分布式文件系统、Web服务器)
CPU
辅存
越往金字塔顶部，存储器
访问速度越来越快
容量越来越小
价格也越来越昂贵

CPU（中央处理器，Central Processing Unit）是计算机系统中负责执行程序指令和数据运算的核心部件。一个典型的 CPU 通常包括以下几个主要组成部分：

1. **控制单元**：控制单元负责从内存中获取待执行的指令并解码、译码。它还负责生成相应的操作命令以调度其他硬件组件（例如寄存器、内存等）。控制单元主要由以下部件构成：
   - 指令寄存器 (Instruction Register)
   - 程序计数器 (Program Counter)
   - 地址寄存器 (Address Register)
   - 指令译码器 (Instruction Decoder)
2. **运算器**：运算器负责处理各种数字信号及逻辑操作，如加法、减法、乘法、除法、移位等。以下部件属于运算器的重要构成：
   - 算术逻辑单元 (Arithmetic and Logic Unit, ALU)
   - 累加寄存器 (Accumulator)
   - 数据缓冲寄存器 (Data Buffer Register)
   - 标志寄存器 (Flag Register)
3. **寄存器集**：寄存器用于存储临时信息，增强 CPU 计算能力。寄存器类型有很多，根据功能可划分为：
   - 通用寄存器
   - 数据寄存器
   - 状态寄存器
   - 特殊功能寄存器（如栈指针或者基址）
4. **高速缓存**：为降低 CPU 与内存之间访问速度的差异，提高效率，部分 CPU 设计中还包含一定空间的高速缓存。它可以暂时存储近期频繁使用的数据或者代码块，便于快速调用。

这些组件共同协同工作，使 CPU 能够有效地执行程序代码、处理各种算术和逻辑任务以及存取数据等。需要注意的是，不同架构的 CPU（例如 RISC 和 CISC）在性能和复杂度上可能会有所差异，具体实现细节亦相应发生变化。同时，CPU 技术在不断演进，随着制程技术的不断改进，CPU 在能源消耗和运行速度方面也正在不断优化。

合肥工业大学计算机学院贾伟

三、 CPU 的寄存器

运算器中的寄存器

暂存寄存器： 用于暂时存放从主存中读取来的数据，这个数据不能放在通用寄存器中，否则会破坏其原有的内容。

累加寄存器（ACC）： 累加寄存器通常简称累加器，他是一个通用寄存器。功能是：当运算器的ALU单元执行算数或逻辑运算时，为ALU提供一个工作其，累加寄存器也暂存ALU运算的结果。一般通用寄存器的数量在16个以上，因此需要在指令格式中对寄存器号加以编址。·

通用寄存器（GPR）： 可以作为某种寻址方式的专用寄存器，主要用于放操作数（源操作数和，目的操作数，以及中间结果）常见的通用寄存器有AX、BX、CX、DX、以及堆栈指针SP（指示栈顶地址）等。

状态条件寄存器（PSW）： 状态条件寄存器，也叫程序状态字寄存器，保存由算术指令和逻辑指令运行和测试的结果建立的各种条件码内容，如运算结果进位标志C，运算结果溢出标志V，运算结果为0标志Z，运算结果为负的标志N，这些标志通常分别由一位触发器保存。

三、 CPU 的寄存器

8.1

控制器中的寄存器

程序计数寄存器（PC）： 为了能够保证程序能够持续不断地执行，CPU必须采取程序计数器这一手段，通常程序计数器又称为指令计数器。在程序开始前，必须将它的起始地址，及程序的第一条指令的所在内存单元的地址送入PC。当执行指令时CPU会自动修改PC的内容，以便使其保存的总要是将要执行的下一条指令地址。但是要注意JUMP类指令取出时，PC仍是先加1，因为此时JUMP指令还未执行，当加1后在执行跳转命令。因此程序计数器是具有寄存信息和计数两种功能。

指令寄存器（IR)： 指令寄存器用来保存当前正在执行的指令（内容不可变），当执行一条指令时，先把它从内存取到数据缓冲寄存器中，然后传至指令寄存器。指令划分为操作码和地址码字段（参见指令系统章节),是以二进制形式存储。为了执行任何给定的指令，必须对操作码进行测试，以便识别所要求的操作。指令译码器就是做这项工作的。指令寄存器中操作码字段的输出就是指令译码器的输入。操作码一经译码后，即可向操作控制器发出具体操作的特定信号。

存储器数据寄存器（MDR）： 主存数据寄存器(MDR)，与主存地址寄存器(MAR)帮助完成CPU和主存储器之间的通信，MDR暂存要被写入地址单元或者从地址单元读出的数据。即暂存从主存读入一条指令或数字，反之，写入前也要有MDR暂存。 **作用：**

- 作为CPU、内存和外部设备之间信息传送的中转站。
- 补偿作为CPU、内存和外部设备之间在操作速度上的差别。
- 在单累加器结构的运算器中，MDR还可作为操作数寄存器。

储存器地址寄存器（MAR）： 用于存放将被访问的存储单元的地址。

三、 CPU 的寄存器

1. 用户可见寄存器

用户可对这类寄存器编程，通过优化使CPU因使用这类寄存器而减少对主存的访问次数

通常CPU执行机器语言访问的寄存器为用户可见寄存器， 按其特征又可分为以下几类：

(1)**通用寄存器。**通用寄存器可由程序设计者指定许多功能，可用于存放操作数，也可作为满足某种寻址方式所需的寄存器。基址寻址所需的基址寄存器、变址寻址所需的变址寄存器和堆栈寻址所需的栈指针，都可用通用寄存器代替。寄存器间接寻址时还可用通用寄存器存放有效地址的地址。

(2)**数据寄存器。**数据寄存器用于存放操作数，其位数应满足多数数据类型的数值范围，有些机器允许使用两个连读的寄存器存放双倍字长的值。还有些机器的数据寄存器只能用于保存数据，不能用于操作数地址的计算。

(3)**地址寄存器。**地址寄存器用于存放地址，其本身可以具有通用性，也可用于特殊的寻址方式，如用于基址寻址的段指针(存放基地址)、用于变址寻址的变址寄存器和用于堆栈寻址的栈指针。地址寄存器的位数必须足够长，以满足最大的地址范围。

(4)**条件码寄存器。**这类寄存器中存放条件码，它们对用户来说是部分透明的。条件码是CPU根据运算结果由硬件设置的位(bits)， 如算术运算会产生正、负、零或溢出等结果。条件码可被测试，作为分支运算的依据。此外，有些条件码也可被设置，如最高位进位标志C，可用指令对它置位和复位。将条件码放到一个或多个寄存器中，就构成了条件码寄存器。

三、 CPU 的寄存器

1. 用户可见寄存器

(1) 通用寄存器　　存放操作数

可作 某种寻址方式所需的 专用寄存器

(2) 数据寄存器　　存放操作数（满足各种数据类型）

两个寄存器拼接存放双倍字长数据

(3) 地址寄存器　　存放地址，其位数应满足最大的地址范围

用于特殊的寻址方式　段基值　栈指针

(4) 条件码寄存器　存放条件码，可作程序分支的依据

如 正、负、零、溢出、进位等

三、 CPU 的寄存器

2. 控制和状态寄存器

另一类属于控制和状态寄存器，用户不可对这类寄存器编程，它们被控制部件使用以控制CPU的操作，也可被带有特权的操作系统程序使用，从而控制程序的执行。

一、控制寄存器

(1) MAR:存储器地址寄存器，用于存放将被访问的存储单元的地址

(2) MDR:存储器数据寄存器，用于存放欲存人存储器中的数据或最近从存储器中读出的数据

(3) PC:程序计数器,存放现行指令的地址,通常具有计数功能。当遇到转移类指令时，PC的值可被修改。

(4) IR：指令寄存器，存放当前欲执行的指令。

通过这4个寄存器，CPU可与主存进行信息交换，例如：从PC取指令地址送至MAR，启动存储器做读操作，存储器就可将指定地址单元内的指令读至MDR，再由MDR送至IR。这4个寄存器中，MAR、MDR、IR对用户不可见，PC对用户可见。

在CPU内部必须给ALU提供数据，因此ALU必须可直接访问MDR和用户可见寄存器，ALU的外围还可以有另一些寄存器，这些寄存器用于ALU的输入/输出以及用于和MDR及用户可见寄存器交换数据。

三、 CPU 的寄存器

二、状态寄存器

状态寄存器用来存放条件码

(1) PSW寄存器：

存放程序状态字PSW，存放条件码和其他状态信息

(2 中断标记寄存器：

在有中断系统的机器中还有中断标记寄存器

合肥工业大学计算机学院贾伟

2. 控制和状态寄存器

(1) 控制寄存器

PC → MAR → M → MDR → IR

控制 CPU 操作

其中 MAR、MDR、IR　　用户不可见

PC　　用户可见

(2) 状态寄存器

状态寄存器　　存放条件码

PSW 寄存器　　存放程序状态字

三、 CPU 的寄存器

不同机器的CPU中，寄存器组织是不一样的，下图画出了Z8000、8086和MC68000三种机器的寄存器组织。



Zilog Z8000有16个16位的通用寄存器，这些寄存器可存放地址、数据，也可作为变址寄存器，其中有两个寄存器被用作栈指针，寄存器可被用作8位和32位的运算。

Z8000中有5个与程序状态有关的寄存器：其中1个用于存放状态标记，2个用于程序计数器，2个用于存放偏移量。确定一个地址需要两个寄存器。

三、 CPU 的寄存器

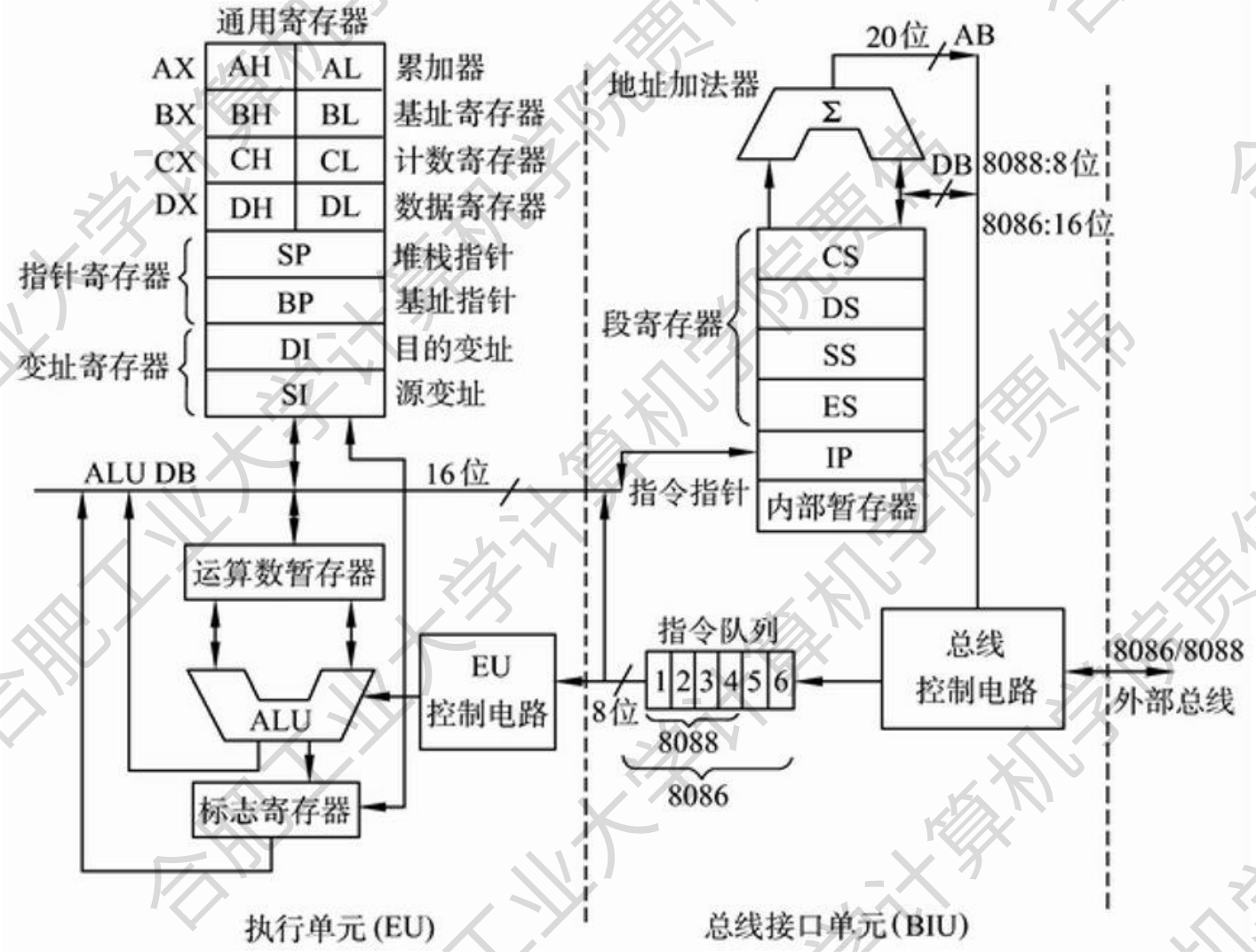


Intel 8086采用不同的寄存器组织，尽管某些寄存器可以通用，但它的每个寄存器大多是专用。

- 它有4个16位的**数据寄存器**，即AX(累加器)，BX（基址寄存器)，CX(计数寄存器)和DX(数据寄存器)，也可兼作8个8位的寄存器(AH、AL、BH、BL、CH、CL、DH、DL)。

- 还有2个16位的**指针**(栈指针SP和基址指针BP)和2个**变址寄存器**(源变址寄存器SI和目的变址寄存器DI)。在一些指令中，寄存器是隐式使用的，如乘法指令总是用累加器。

- 8086还有4个**段地址寄存器**(代码段CS，数据段DS，堆栈段SS和附加段ES)
- 指令指针IP(相当于PC)和状态标志寄存器F。



8086 CPU 中寄存器总共为 14 个，且均为 16 位。

即 AX，BX，CX，DX，SP，BP，SI，DI，IP，FLAG，CS，DS，SS，ES 共 14 个。而这 14 个寄存器按照一定方式又分为了通用寄存器，控制寄存器和段寄存器。

2. 控制和状态寄存器

AX，BX，CX，DX称作为数据寄存器：
AX (Accumulator)：累加寄存器，也称之为累加器；
BX (Base)：基地址寄存器；
CX (Count)：计数器寄存器；
DX (Data)：数据寄存器；

SP和BP又称作为指针寄存器：
SP (Stack Pointer)：堆栈指针寄存器；
BP (Base Pointer)：基指针寄存器；

SI和DI又称作为变址寄存器：
SI (Source Index)：源变址寄存器；
DI (Destination Index)：目的变址寄存器；

控制寄存器：
IP (Instruction Pointer)：指令指针寄存器；
FLAG：标志寄存器；

段寄存器：
CS (Code Segment)：代码段寄存器；
DS (Data Segment)：数据段寄存器；
SS (Stack Segment)：堆栈段寄存器；
ES (Extra Segment)：附加段寄存器；

三、 CPU 的寄存器



Motorola MC68000的寄存器组织介于Zilog和Intel微处理器之间，它将32位寄存器分为8个数据寄存器(D0～D7)和9个地址寄存器(A0～A7′)。

- 数据寄存器主要用于数据运算，当需要变址时，也可作变址寄存器用。寄存器允许8位、16位和32位的数据运算，这由操作码确定。
- 地址寄存器存放32位地址(没有段)，其中两个(A7和A7′)也可用作堆栈指针，分别供用户和操作系统使用。针对当前执行的模式，这两个寄存器在某个时刻只能用一个。
- 此外，MC68000还有一个32位的程序计数器PC和一个16位的状态寄存器。

与Zilog的设计者类似，Motorola设计的寄存器组织也不含专用寄存器。

四、 控制单元 CU 和中断系统 8.1

控制单元CU是提供完成机器全部指令操作的微操作命令序列部件

现代计算机中微操作命令序列的形成方法有两种:
- 一种是组合逻辑设计方法，为硬联线逻辑;
- 另一种是微程序设计方法，为存储逻辑。

中断系统主要用于处理计算机的各种中断。

四、 控制单元 CU 和中断系统 8.1

1. 控制单元CU 产生全部指令的微操作命令序列

组合逻辑设计 硬连线逻辑

微程序设计 存储逻辑

参见 第 4 篇

2. 中断系统

参见 8.4 节

五、ALU

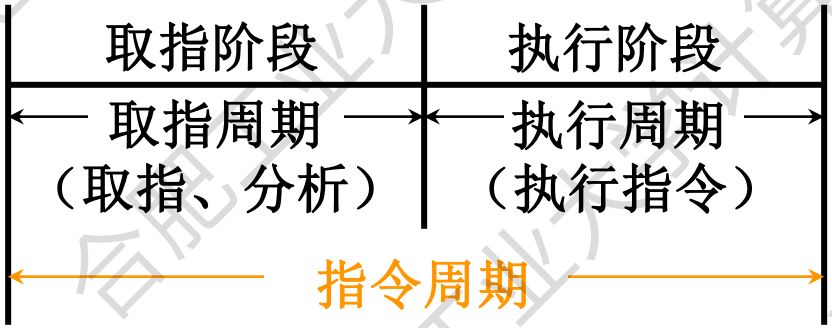
参见 第 6 章

一、 指令周期的基本概念

1．指令周期

CPU取出并执行一条指令所需的全部时间，也即CPU完成一条指令的时间。

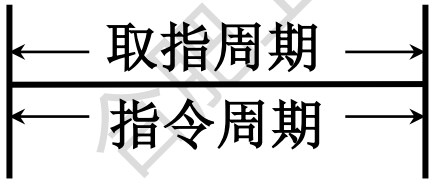
完成一条指令 { 取指、分析 —— 取指周期
执行 —— 执行周期



2. 每条指令的指令周期不同

2.由于各种指令操作功能不同，因此各种指令的指令周期是不相同的

■无条件转移指令"JMP X"，在执行阶段不需要访问主存，而且操作简单，完全可以在取指阶段的后期将转移地址X送至PC，以达到转移的目的。这样，“JMP X"指令的指令周期就是取指周期。

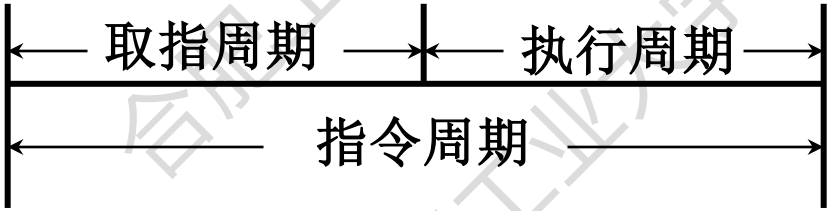


NOP

2. 每条指令的指令周期不同

2.由于各种指令操作功能不同，因此各种指令的指令周期是不相同的

■又如一地址格式的加法指令"ADD X”，在执行阶段首先要从X所指示的存储单元中取出操作数，然后和ACC的内容相加，结果存于ACC，故这种指令的指令周期在取指和执行阶段各访问一次存储器，其指令周期就包括两个存取周期。

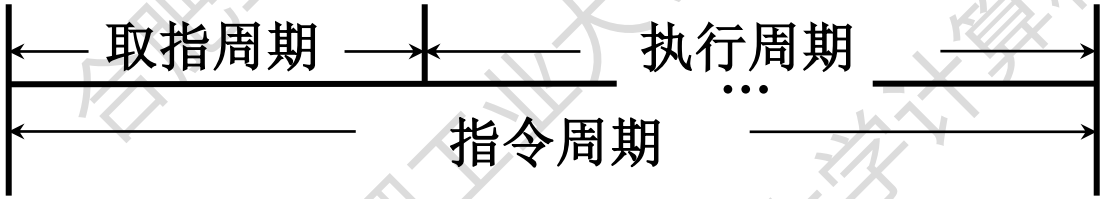


ADD mem

2. 每条指令的指令周期不同

2.由于各种指令操作功能不同，因此各种指令的指令周期是不相同的

■再如乘法指令，其执行阶段所要完成的操作比加法指令多得多，故它的执行周期超过了加法指令。

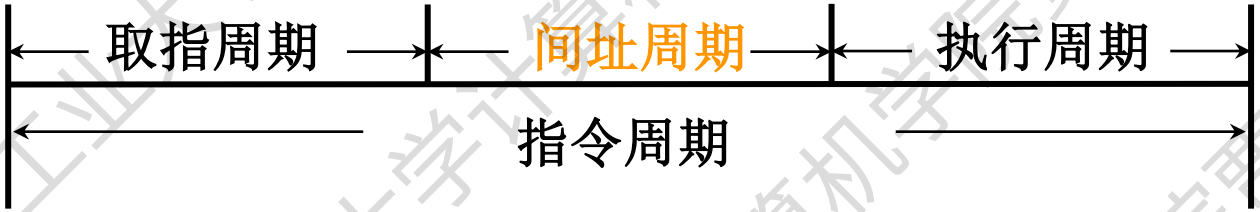


MUL mem

3. 具有间接寻址的指令周期 8.2

3. 具有间接寻址的指令周期

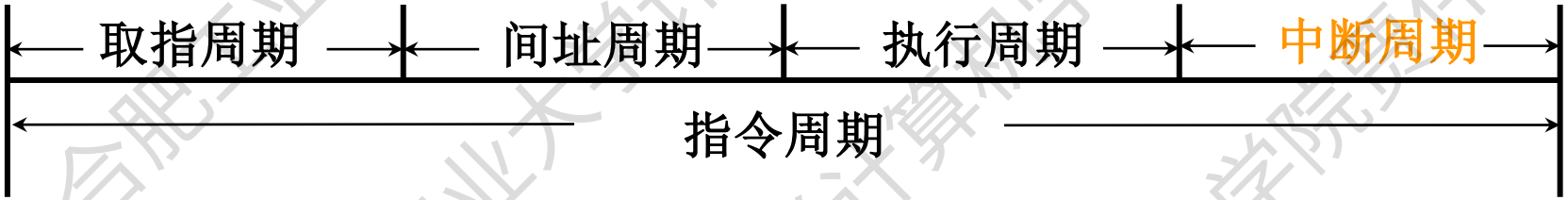
由于指令字中只给出操作数有效地址的地址，因此，为了取出操作数，需先访问一次存储器，取出有效地址，然后再访问存储器，取出操作数。这样，间接寻址的指令周期就包括取指周期、间址周期、和执行周期3个阶段。



4.带有中断周期的指令周期

4. 带有中断周期的指令周期

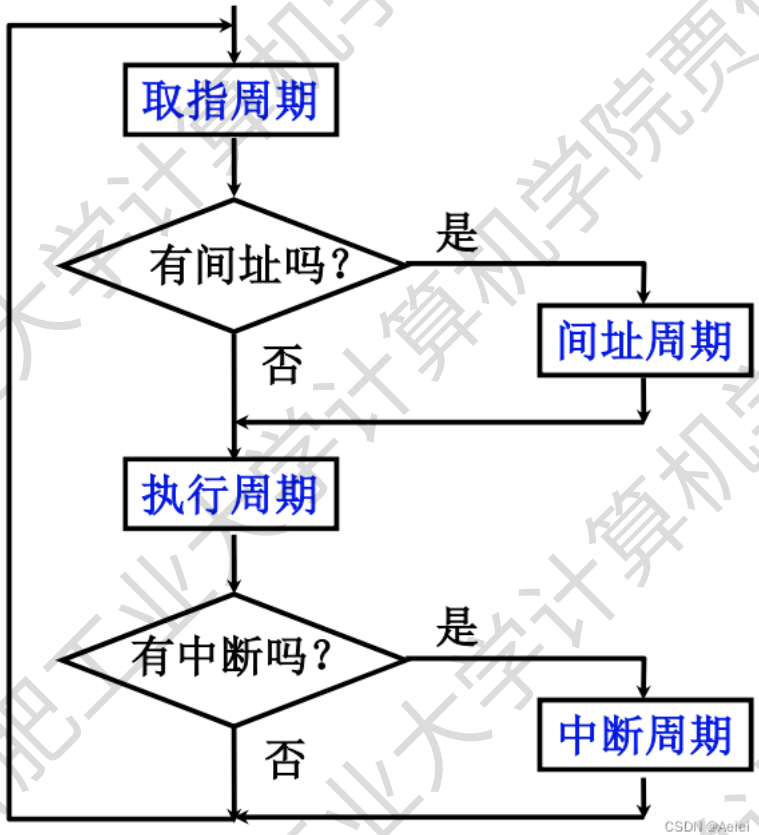
当CPU采用中断方式实现主机与I/O设备交换信息时，CPU在每条指令执行阶段结束前，都要发中断查询信号，以检测是否有某个I/O设备提出中断请求。如果有请求,，CPU则要进入中断响应阶段，又称中断周期。在此阶段，CPU必须将程序断点保存到存储器中。这样，一个完整的指令周期应包括取指、间址、执行和中断4个子周期。



5. 指令周期流程

5. 指令周期流程

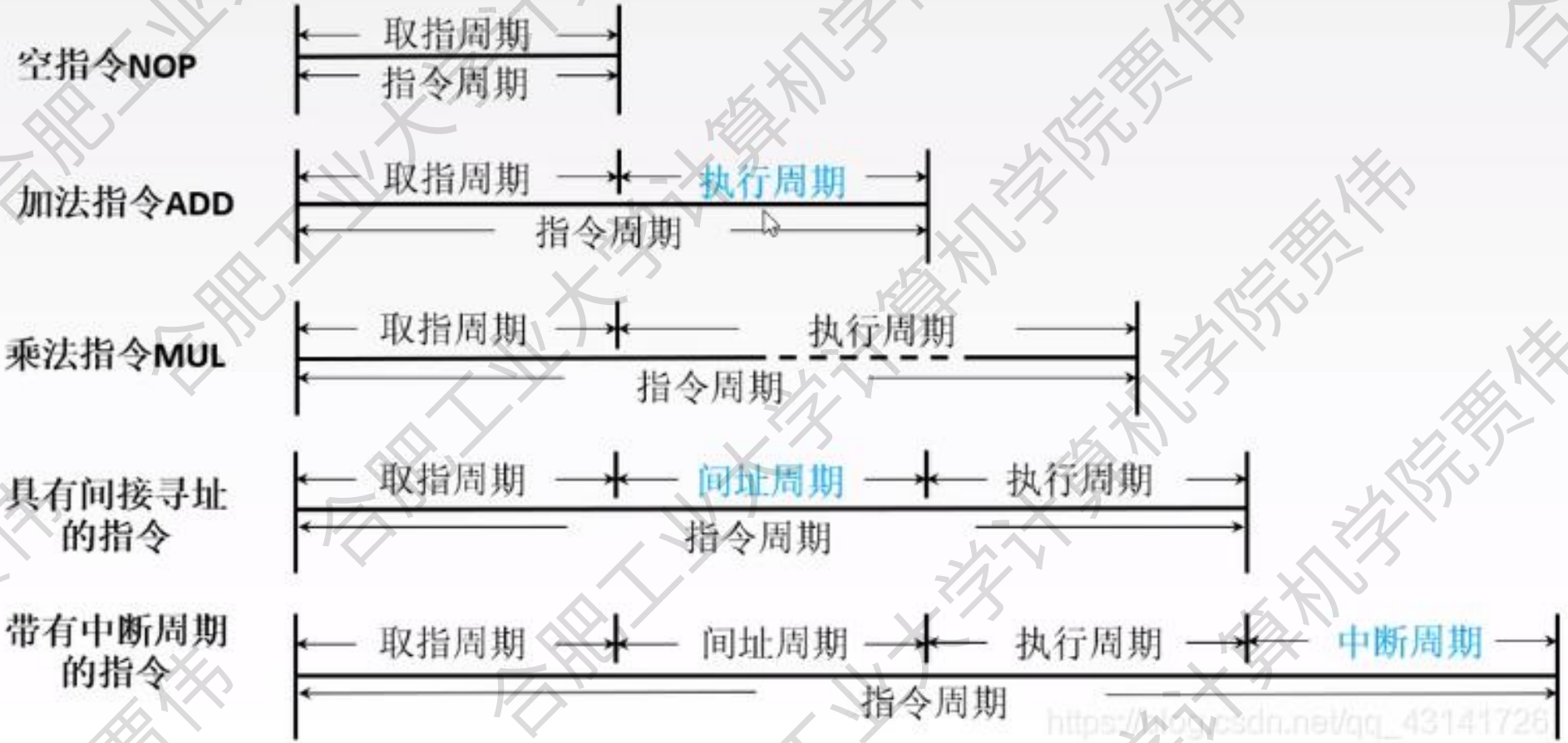
由于间址周期和中断周期不一定包含在每个指令周期内，故流程图中用菱形框判断。



合肥工业大学计算机学院贾伟

指令周期由若干个机器周期/cpu周期组成，下面的图为不同的指令的指令周期

每个指令周期内机器周期数可以不等，每个机器周期内的节拍数也可以不等。



6. CPU 工作周期的标志

CPU 访存有四种性质

上述4个周期都有CPU访存操作：
★ 取指周期：取指令
★ 间址周期：取有效地址
★ 执行周期：取有操作数
★ 中断周期：存程序地址
这4个周期又可称为CPU的工作周期，为了区分它们，在CPU内可设置4个标志触发器。

CPU 的

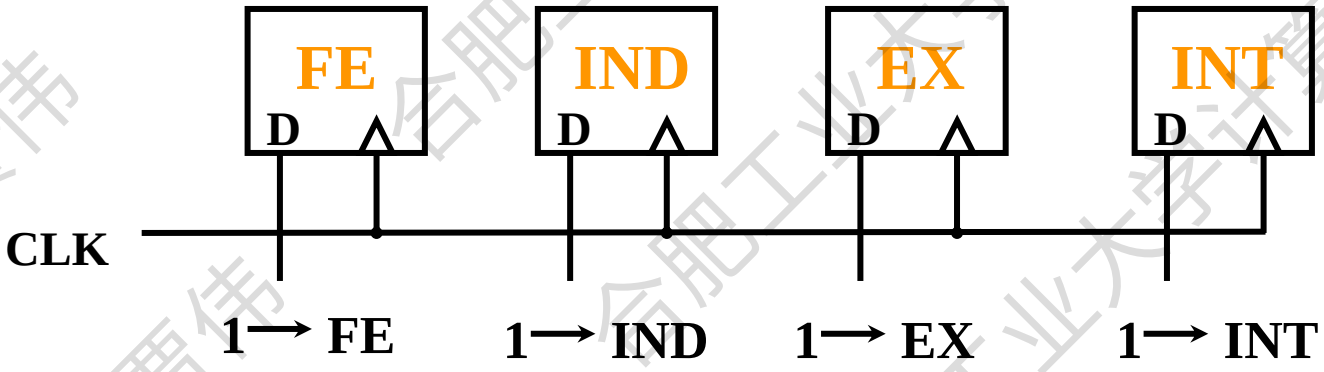
4个工作周期

取 指令 取指周期

取 地址 间址周期

取 操作数 执行周期

存 程序断点中断周期



FE-The Fetch Cycles

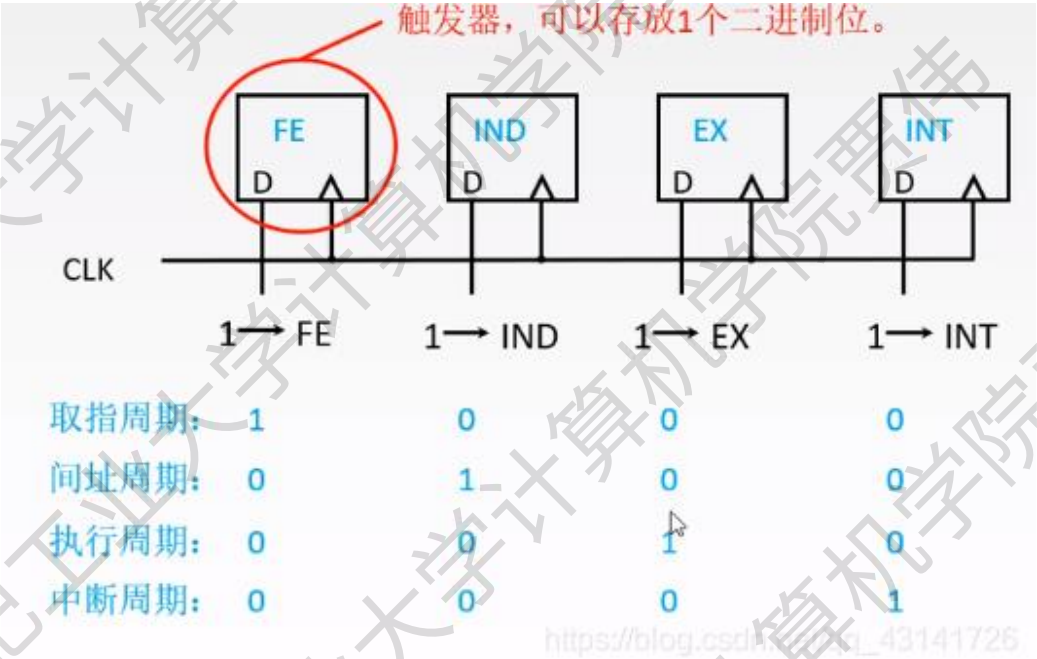
IND-The Indirect Cycles

EX-The Execute Cycles

INT-The Interrupt Cycles

6. CPU 工作周期的标志

如何判断在哪个阶段呢，提供一个硬件结构:触发器



一个方框代表一个触发器,4个代表4个不同的周期。FE，IND，EX，INT，分别代表的是取指周期,间指周期,执行周期,间断周期

6. CPU 工作周期的标志

设置CPU 工作周期标志触发器对设计控制单元十分有利：

★ 在取指阶段，只要设置取指周期标志触发器FE为1，由它控制取指阶段的各个操作，便获得对任何一条指令的取指命令序列。

★ 在间接寻址时，间址次数可由间址周期标志触发器IND确定，当它为"0"状态时，表示间接寻址结束。

★ 对于一些执行周期不访存的指令(如转移指令、寄存器类型指令)，同样可以用它们的操作码与取指周期标志触发器的状态相"与"，作为相应微操作的控制条件。

合肥工业大学计算机学院贾伟

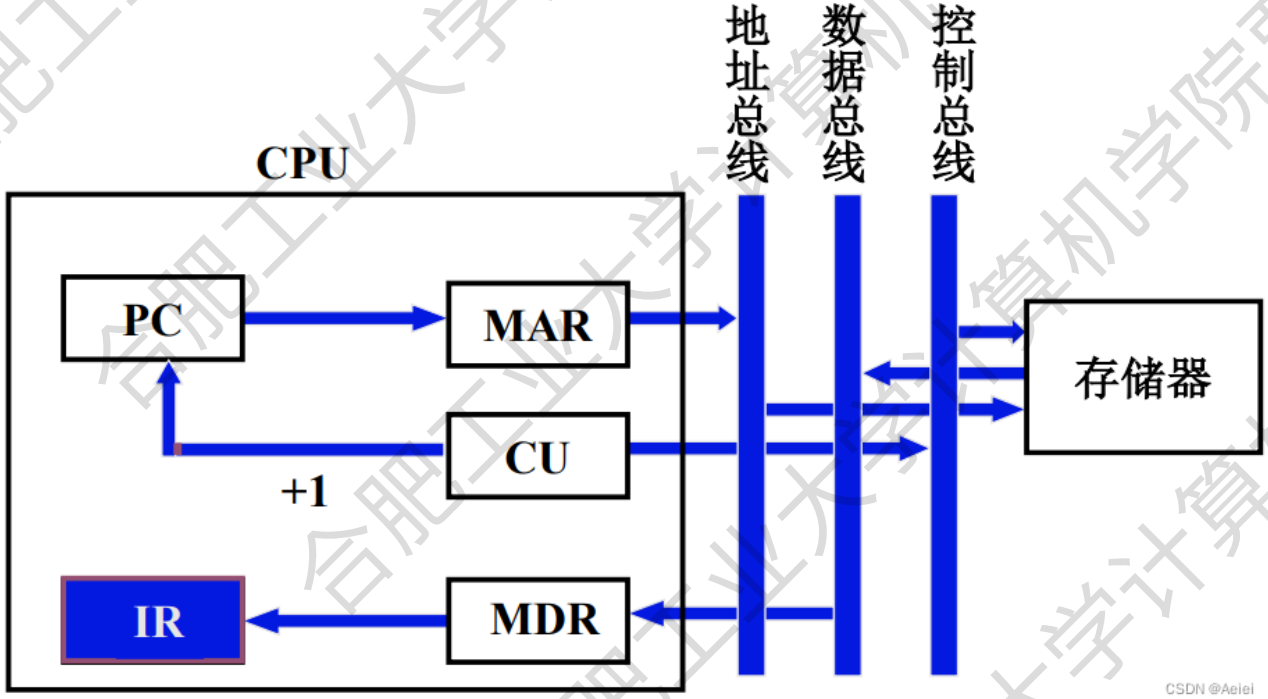
二、 指令周期的数据流

8.2

为了便于分析指令周期中的数据流，假设 CPU中有存储器地址寄存器MAR、存储器数据寄存器MDR、程序计数器PC和指令寄存器IR

1. 取指周期数据流

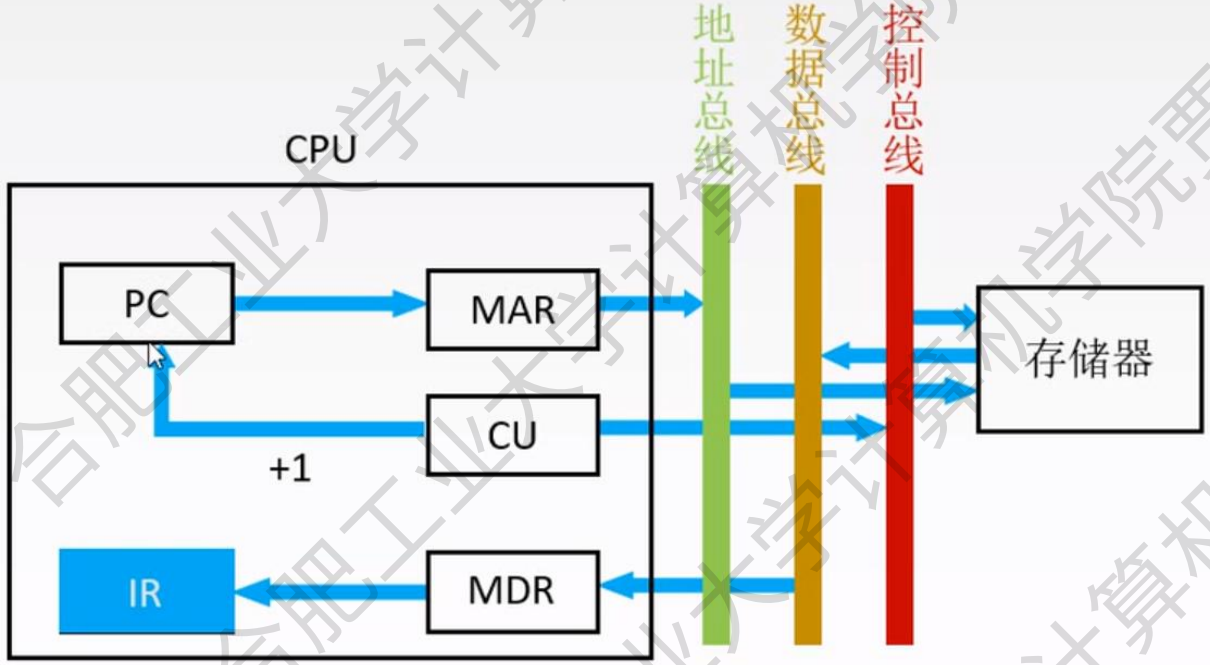
下图所示的是取指周期的数据流。PC中存放现行指令的地址，该地址送到MAR并送至地址总线，然后由控制部件CU向存储器发读命令，使对应MAR所指单元的内容(指令)经数据总线送至MDR，再送至IR，并且CU控制PC内容加1，形成下一条指令的地址。



CSDN @Aelei

二、指令周期的数据流

8.2



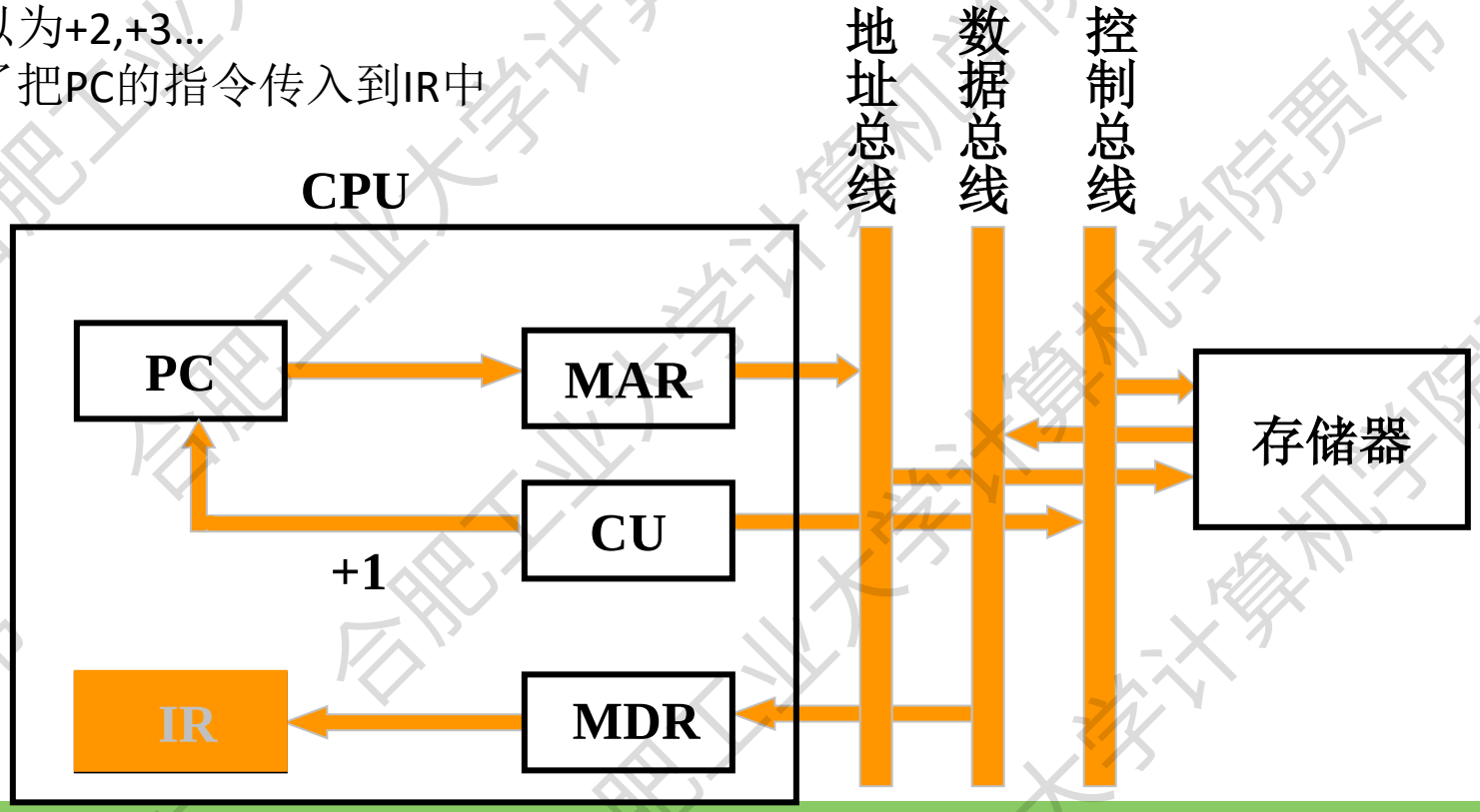
1. 当前指令地址送至存储器地址寄存器，记做：(PC) → MAR
2. CU发出控制信号，经控制总线传到主存，这里是读信号，记做：1 → R
3. 将MAR所指主存中的内容经数据总线送入MDR，记做：M(MAR) → MDR
4. 将MDR中的内容(此时是指令)送入IR，记做：(MDR) → IR
5. CU发出控制信号，形成下一条指令地址，记做：(PC)+1 → PC

1.(PC)->MAR
2.CU发出控制信号,读信号:1->R表示CU发送信号到存储器
3.M(MAR)->MDR
4.(MDR)->IR
5.(PC)+1->PC
这里的+1可以为+2,+3…
目的就是为了把PC的指令传入到IR中

二、 指令周期的数据流

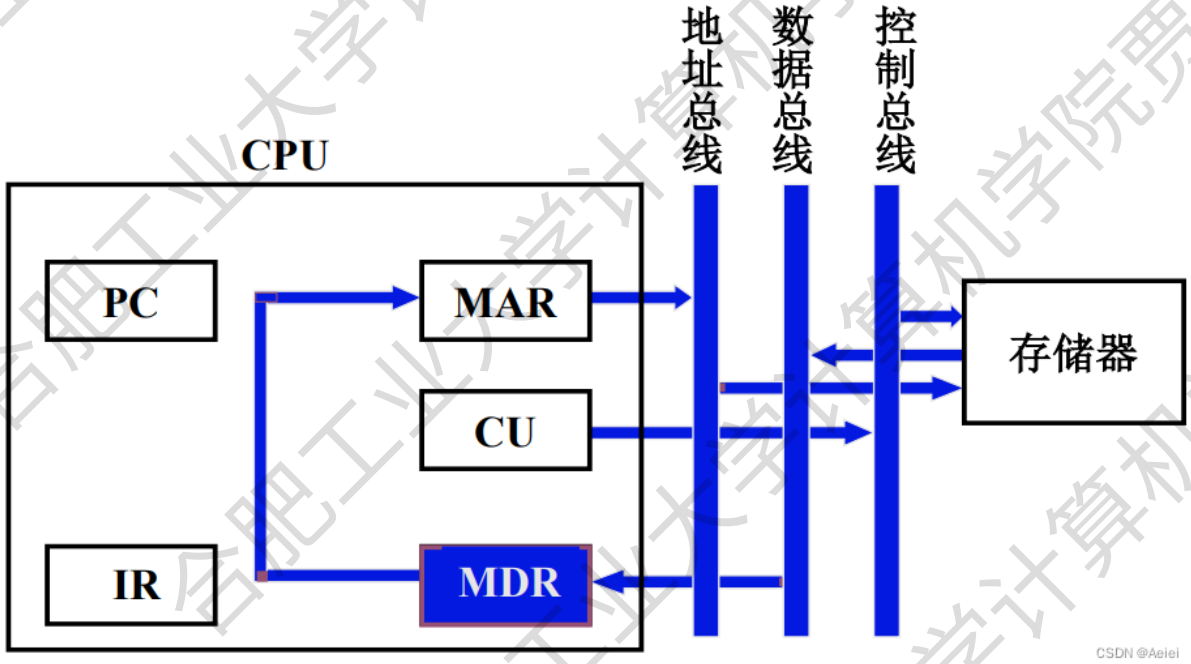
1. 取指周期数据流

1.(PC)->MAR
2.CU发出控制信号,读信号:1->R表示CU发送信号到存储器
3.M(MAR)->MDR
4.(MDR)->IR
5.(PC)+1->PC
这里的+1可以为+2,+3…
目的就是为了把PC的指令传入到IR中

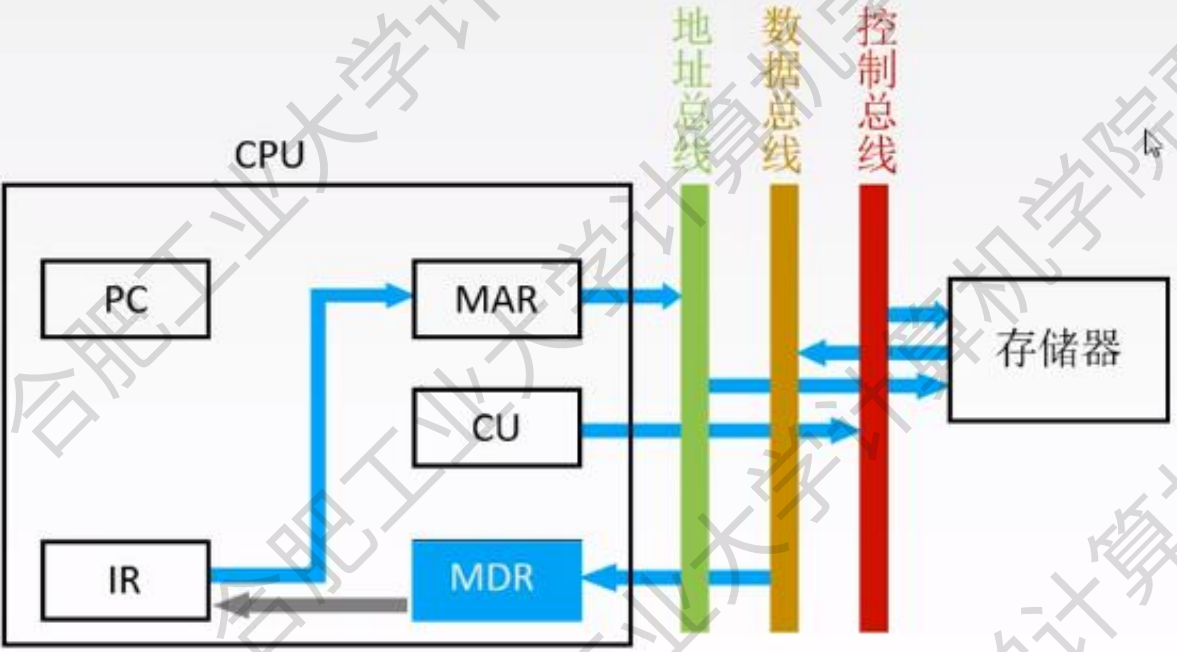


2. 间址周期数据流

间址周期的数据流如下图所示。一旦取指周期结束，CU便检查IR中的内容，以确定其是否有间址操作。如果需要间址操作，则MDR中指示形式地址的右N位(MDR中存有指令，指令中有操作码在前和操作数在后，操作数所在的后N位（记作Ad(MDR))将被送到MAR，又送至地址总线，此后CU向存储器发读命令，以获取有效地址并存至MDR。



2. 间址周期数据流

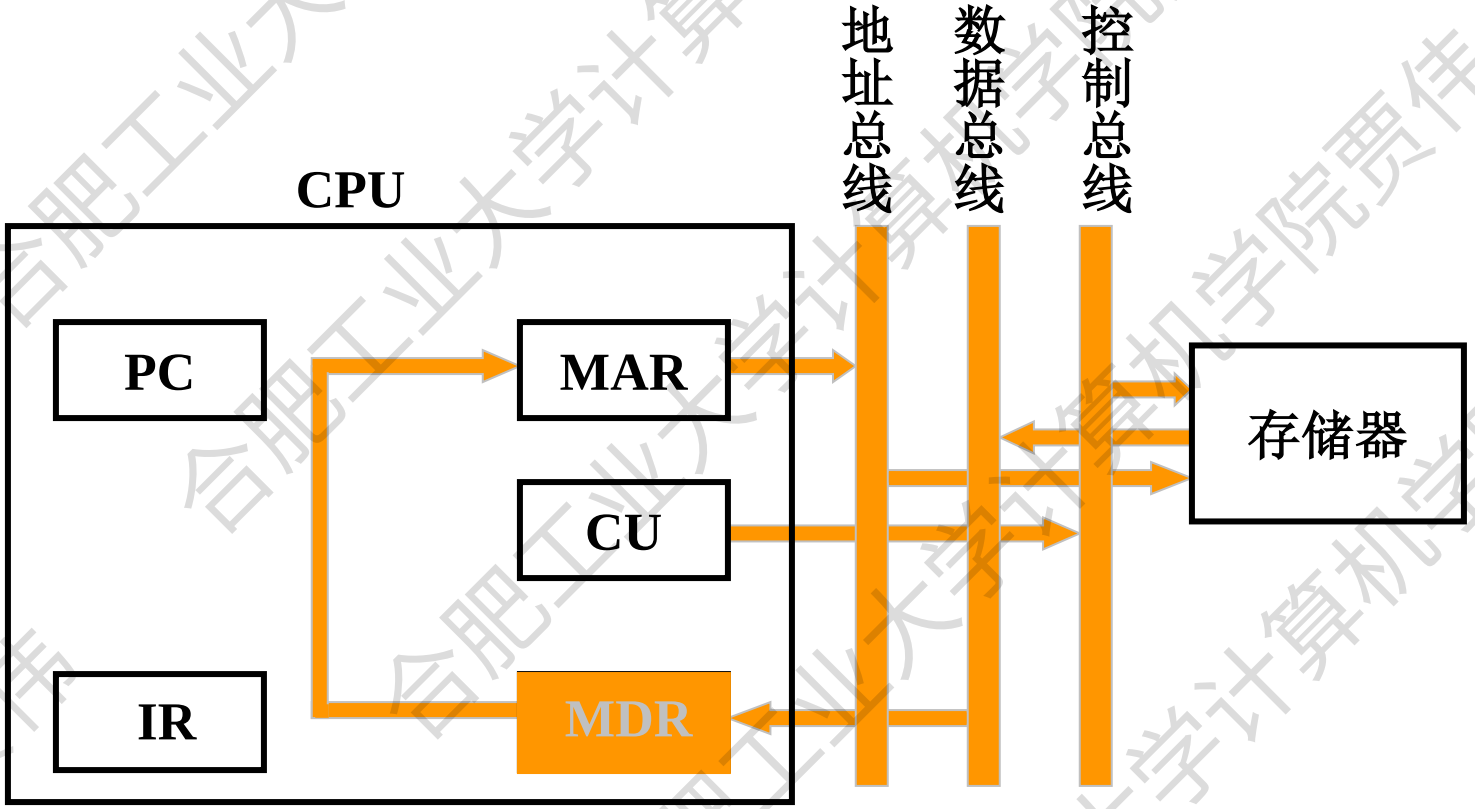


1. 将指令的地址码送入MAR，
记做：Ad(IR) → MAR
或Ad(MDR) → MAR
2. CU发出控制信号，启动主存做读操作，
记做：1 → R
3. 将MAR所指主存中的内容经数据总线送入MDR，记做：M(MAR) → MDR
4. 将有效地址送至指令的地址码字段，
记做：MDR→ Ad(IR)

2. 间址周期数据流

8.2

1.从IR中将指令地址传入到MAR
Ad(IR)->MAR
2.CU发出控制信号,读信号:1->R
3.MAR中的内容到MDR中:M(MAR)->MDR
4.将有效地址送至指令的地址码字段:MDR->Ad(IR)
或者直接在MDR中操作。不同的第4步代表着不同的下一个数据流的指向



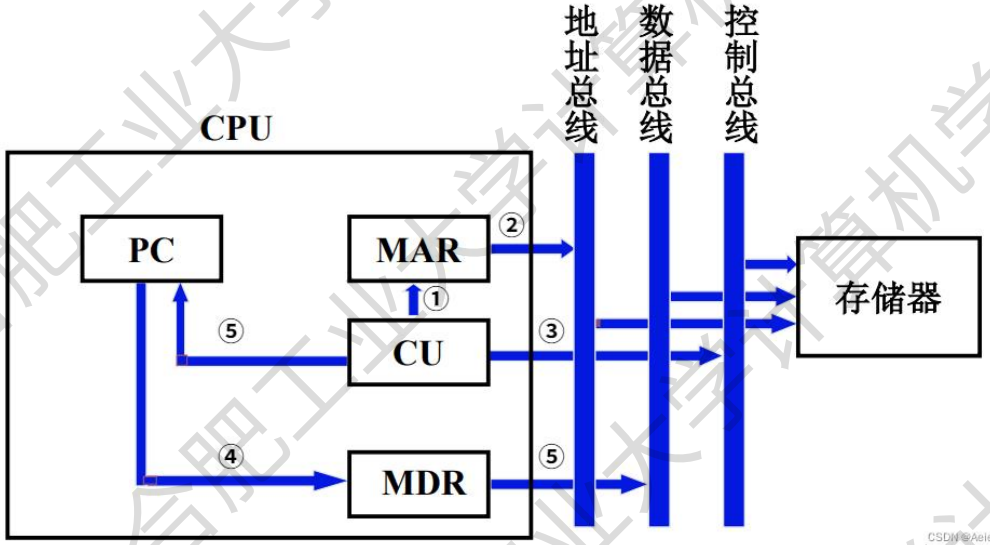
3. 执行周期数据流

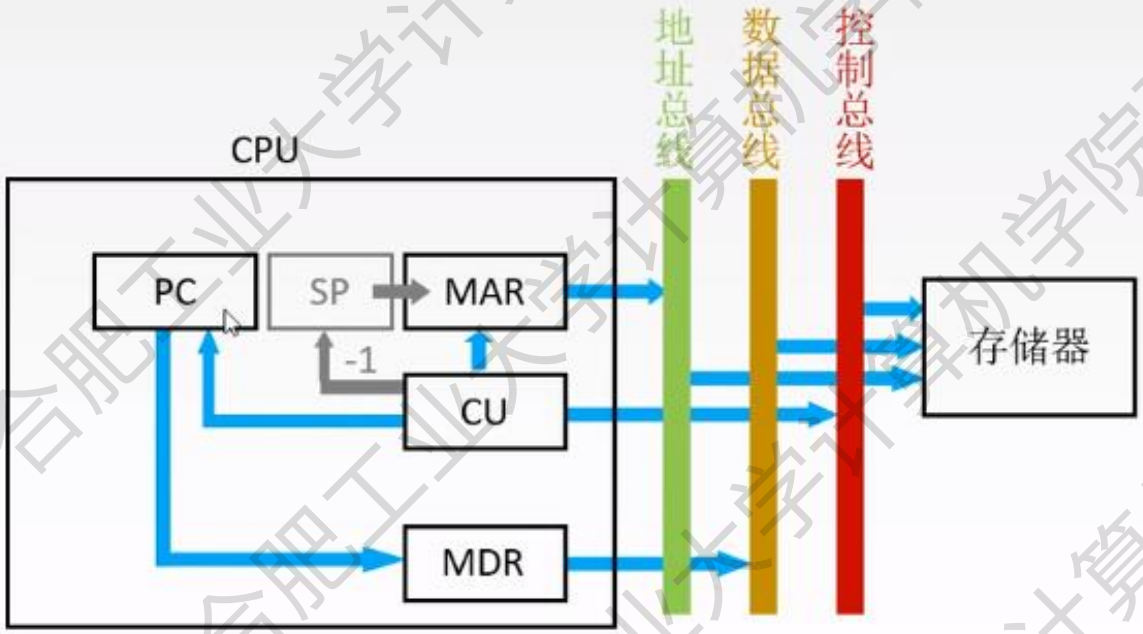
由于不同的指令在执行周期的操作不同，因此执行周期的数据流是多种多样的，可能涉及CPU内部寄存器间的数据传送、对存储器(或I/O)进行读写操作或对ALU的操作，因此，无法用统一的数据流图表示。

合肥工业大学计算机学院贾伟

4．中断周期数据流

CPU进入中断周期要完成一系列操作，其中PC当前的内容必须保存起来，以待执行完中断服务程序后可以准确返回到该程序的间断处，这一操作的数据流如下图所示。图中由CU把用于保存程序断点的特殊存储器地址(如栈指针的内容)送往MAR，并送到地址总线上，然后由CU向存储器发写命令，并将PC的内容(程序断点)送到MDR，最终使程序断点经数据总线存入存储器。此外，CU还需将中断服务程序的人口地址送至PC，为下一个指令周期的取指周期做好准备。





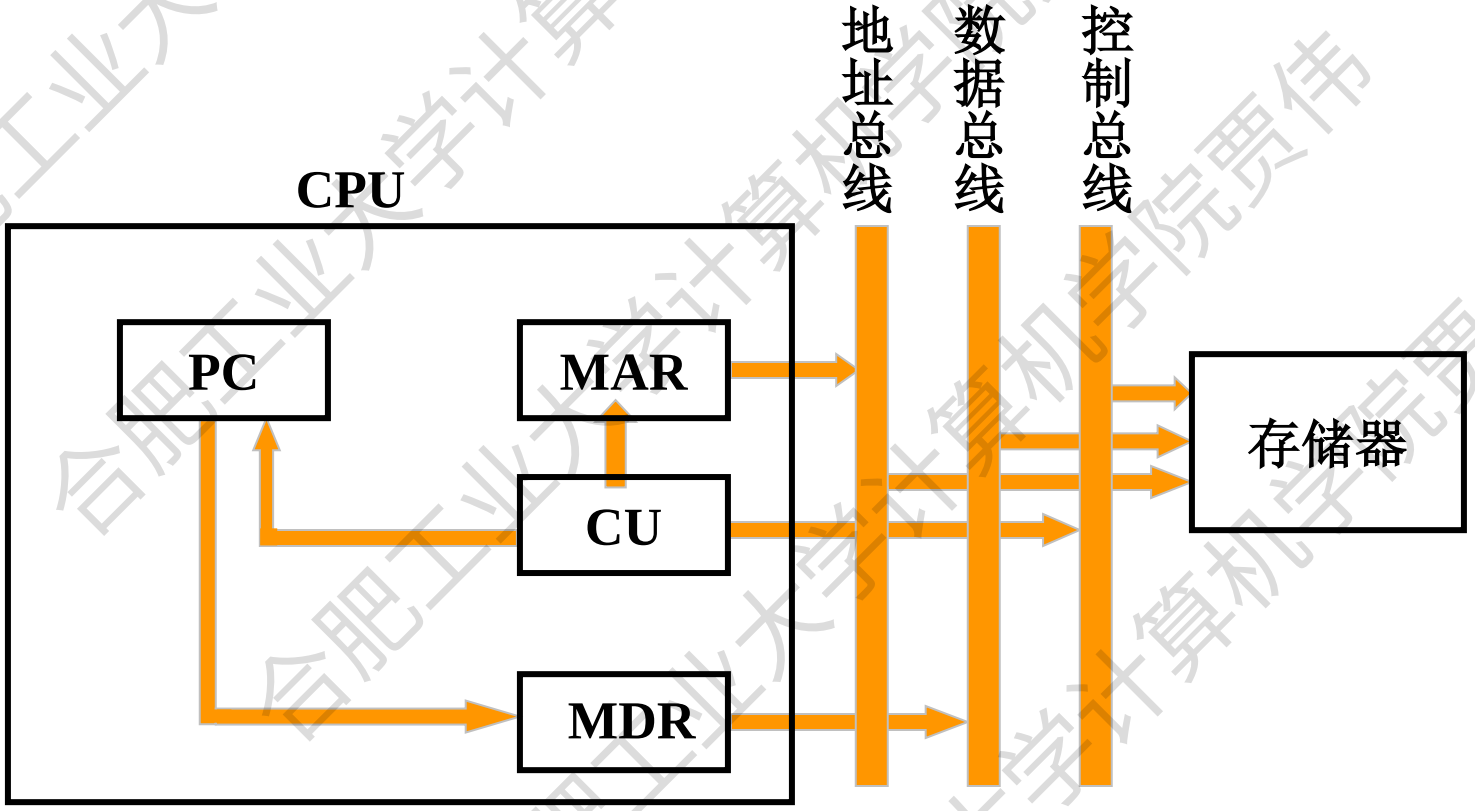
中断：暂停当前任务去完成其他任务。
为了能够恢复当前任务，需要**保存断点**。
一般使用堆栈来保存断点，这里用SP表示栈顶地址，假设**SP指向栈顶元素**，进栈操作是**先修改指针，后存入数据**。

1. CU控制将SP减1，修改后的地址送入MAR
记做：(SP)-1 → SP，(SP) → MAR
本质上是将断点存入某个存储单元，假设其地址为a，故可记做：a → MAR
2. CU发出控制信号，启动主存做**写操作**，
记做：1 → W
3. 将断点(PC内容) 送入MDR，
记做：(PC) → MDR
4. CU控制将中断服务程序的入口地址(由向量地址形成部件产生)送入PC，
记做：向量地址→ PC

中断周期的数据流:
中断:暂停当前的任务去完成其他的任务
为了可以快速的恢复当前任务,设置断点
一般使用堆栈来保存断定,这里为SP为堆栈表示,这里SP为堆栈的首地址
进栈操作为先修改指针,然后加入数据
进栈为-1,出栈为+1

4．中断周期数据流

CPU进人中断周期要完成一系列操作，其中PC当前的内容必须保存起来，以待执行完中断服务程序后可以准确返回到该程序的间断处，这一操作的数据流如下图所示。图中由CU把用于保存程序断点的特殊存储器地址(如栈指针的内容)送往MAR，并送到地址总线上，然后由CU向存储器发写命令，并将PC的内容(程序断点)送到MDR，最终使程序断点经数据总线存入存储器。此外，CU还需将中断服务程序的人口地址送至PC，为下一个指令周期的取指周期做好准备。



8.3 指令流水

一、如何提高机器速度

1. 提高访存速度

   高速芯片　　Cache　　多体并行

2. 提高 I/O 和主机之间的传送速度

   中断　　DMA　　通道　　I/O 处理机　　多总线

3. 提高运算器速度

   高速芯片　　改进算法　　快速进位链

- **提高整机处理能力**

  高速器件　　改进系统结构，开发系统的并行性

合肥工业大学计算机学院贾伟

指令流水 8.3

1 提高访存速度多体并行：

对多个存储体进行交叉访问，在一个主存周期中，有多个存储体向cpu提供数据，或 cpu 在一个主存周期中访问多个存储器；

2 提高IO和主机之间的传送速度

起初，I/O和主机之间是采用程序控制方式，该方式下，CPU与外设直接不能并行工作；

- 2.1 中断： cpu和外部设备可以在部分实现中并行工作，提高了cpu使用率；
- 2.2 DMA： 直接主存访问；用DMA控制器直接在内存和外设之间进行数据传送；减少cpu在数据传送过程中的占用率；进而提高cpu的使用率；
- 2.3 通道： DMA可以划分为多个子通道（netty的实现原理）；子通道间可以同时准备数据，但传送需要串行占用cpu周期，进一步提高了IO外设与内存间的数据传输率；
- 2.4 IO处理机：内存和IO设备间的数据传输，整个传输过程和数据校验过程，完全由IO处理器来做，cpu被独立出来和解放出来，效率会更高 ；
- 2.5 多总线：

指令流水

8.3

为了提高访存速度，一方面要提高存储器的芯片的性能，另一方面从体系结构上， 如采用多体、Cache等分级存储措施来提高存储器的性能/价格比。 为了进一步提高整机的处理能力，通常可从两方面入手。

(1)提高器件的性能。提高器件的性能一直是提高整机性能的重要途径，计算机的发展史就是按器件把计算机分为电子管、晶体管、集成电路和大规模集成电路这四代的。器件的每一次更新换代都使计算机的软硬件技术和计算机性能获得突破性进展。特别是大规模集成电路的发展，由于其集成度高、体积小、功耗低、可靠性高、价格便宜等特点，使人们可采用更复杂的系统结构造出性能更高、工作更可靠、价格更低的计算机。但是由于半导体器件的集成度越来越接近物理极限，机器速度的提高越来越馒。

合肥工业大学计算机学院贾伟

指令流水

8.3

(2)改进系统的结构，开发系统的并行性。所谓并行包含同时性和并发性两个方面。前者是指两个或多个事件在同一时刻发生，后者是指两个或多个事件在同一时间段发生。也就是说，在同一时刻或同一时间段内完成两种或两种以上性质相同或不同的功能，只要在时间上互相重叠，就存在并行性。

并行性体现在不同等级上。通常分为4个级别：作业级或程序级、任务级或进程级、指令之间级和指令内部级。前两级为粗粒度，又叫过程级，后两级为细粒度，又叫指令级。粗粒度并行性一般用算法(软件)实现，细粒度并行性一般用硬件实现。

从计算机体系上看，粗粒度并行性是在多个处理机上分别运行多个进程，由多台处理机合作完成一个程序；细粒度并行性是指在处理机的操作级和指令级的并行性，其中指令的流水作业就是一项重要技术。这里只讨论有关指令流水的一些主要问题

操作系统的特征

四个基本特征：并发、共享、虚拟、异步

1. 并发

并发性：两个或多个事件在同一时间间隔内发生(最重要的特征)

并行性：两个或多个事件在同一时刻发生

并发性是OS最重要的特征，OS所有的内容都围绕它展开！！

程序不能并发执行，为使多个程序并发执行，引入进程。

二、系统的并行性 8.3

1. 并行的概念

并行 { 并发 两个或两个以上事件在 同一时间段 发生
同时 两个或两个以上事件在 同一时刻 发生

时间上互相重叠

2. 并行性的等级

过程级（程序、进程）	粗粒度	软件实现
指令级（指令之间） （指令内部）	细粒度	硬件实现

CSDN @PacosonSWJTU

指令流水

8.3

- 指令流水类似于工厂的装配线，装配线利用了产品在装配的不同阶段其装配过程不同这一特点，使不同产品处在不同的装配段上，即每个装配段同时对不同产品进行加工，这样可大大提高装配效率。将这种装配生产线的思想用到指令的执行上，就引出了指令流水的概念。

- 从上面分析可知，完成一条指令实际上也可分为许多阶段。为简单起见，把指令的处理过程分为取指令和执行指令两个阶段，在不采用流水技术的计算机里，一旦取指令和执行指令是周而复始地重复出现，各条指令是按顺序串行执行的，如下图所示。

取指令1	执行指令1	取指令2	执行指令2	取指令3	执行指令3	……

合肥工业大学计算机学院贾伟

指令流水

- 图中取指令的操作可由指令部件完成，执行指令的操作可由执行部件完成。进一步分析发现，这种顺序执行虽然控制简单，但这执行中各部件的利用率不高，如指令部件工作时，执行部件基本空闲，而执行部件工作时，指令部件基本空闲。如果指令执行阶段不访问主存，则完全可以利用这段时间取下一条指令，这样就使取下一条指令的操作和执行当前指令的操作同时进行，如下图所示，这就是两条指令的重叠，也即指令的二级流水。



合肥工业大学计算机学院贾伟

三、指令流水原理

1. 指令的串行执行



取指令　取指令部件　完成　总有一个部件 空闲

执行指令　执行指令部件　完成

2. 指令的二级流水



若 取指 和 执行 阶段时间上 完全重叠

指令周期 减半　速度提高 1 倍

CSDN @PacosonSWJTU

指令流水

8.3

由指令部件取出一条指令，并将它暂存起来，如果执行部件空闲，就将暂存的指令传给执行部件执行。与此同时，指令部件又可取出下一条指令并暂存起来，这就叫**指令预取**。显然，这种工作方式能加速指令的执行。如果取指和执行阶段在时间上完全重叠，相当于指令周期将减半。然而进一步分析流水线，就会发现存在两个原因使得执行效率加倍是不可能的。

①指令的执行时间一般大于取指时间，因此，取指阶段可能要等待一段时间，也即存放在指令部件缓冲区的指令还不能立即传给执行部件，缓冲区不能空出。

②当遇到条件转移指令时，下一条指令是不可知的，因为必须等到执行阶段结束后，才能获知条件是否成立，从而决定下条指令的地址，造成时间损失。

通常为了减少时间损失，采用猜测法，即当条件转移指令从取指阶段进入执行阶段时，指令部件仍按顺序预取下一条指令，这样，如果条件不成立，转移没发生，则没有时间损失；若条件成立，转移发生；则所取的指令必须丢掉，并再取新的指令。

合肥工业大学计算机学院贾伟

3. 影响指令流水效率加倍的因素 8.3

(1) 执行时间 > 取指时间



(2) 条件转移指令 对指令流水的影响

必须等 上条 指令执行结束，才能确定 下条 指令的地址，

造成时间损失　　猜测法

解决办法 ？

因素1 执行时间 大于 取指时间；
因素2 条件转移指令对指令流水的影响：
因为只有当指令执行后，才知道结果，根据结果判断是否转移；如果转移的话，将要执行的指令就不是按序的下一条指令；

指令流水

8.3

尽管这些因素降低了两级流水线的潜在效率，但还是可以获得一定程度的加速。为了进一步提高处理速度，需将指令的处理过程分解为更细的几个阶段。

- 取指（FI）：从存储器取出一条指令并暂时存入指令部件的缓冲区。
- 指令译码（DI）：确定操作性质和操作数地址的形成方式。
- 计算操作数地址（CO）：计算操作数的有效地址，涉及到寄存器间址、间址、变址、基址、相对寻址等各种地址计算方式。
- 取操作数（FO）： 从存储器中取操作数(若操作数在寄存器中，则无需此阶段)。
- 执行指令（EI）：执行指令所需的操作，并将结果存于目的位置(寄存器中)。
- 写操作数（WO）：将结果存入存储器。

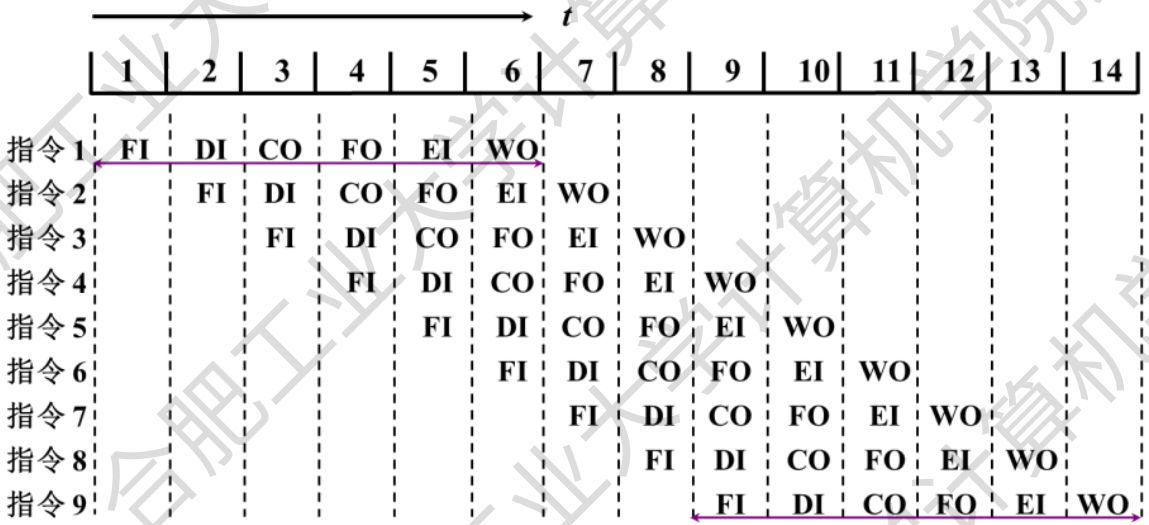
为了说明方便起见，假设上述各段的时间都是相等的，于是可得下图所示的指令六级流水时序。在这个流水线中，处理器有六个操作部件，同时对六条指令进行加工，加快了程序的执行速度。

指令周期被分为6个阶段：

1)FI：取指令；fetch instruction
2)DI：指令译码； decode instruction
3)CO： 形成操作数地址；
4)FO： 取操作数； fetch operand
5)EI： 执行； execute instruction
6)WO： 结果写回；write operand

4. 指令的六级流水

8.3



完成 一条指令 6 个时间单位

串行执行 6 × 9 = 54 个时间单位

六级流水 14 个时间单位

由上图可以看出流水线的执行性能优于串行执行：

完成一条指令需要6个时间单位；

1.串行执行，则完成9条指令，需要**54**个时间单位；

2.采用六级流水线执行，完成9条指令，仅需要**14**个时间单位；

指令流水

8.3

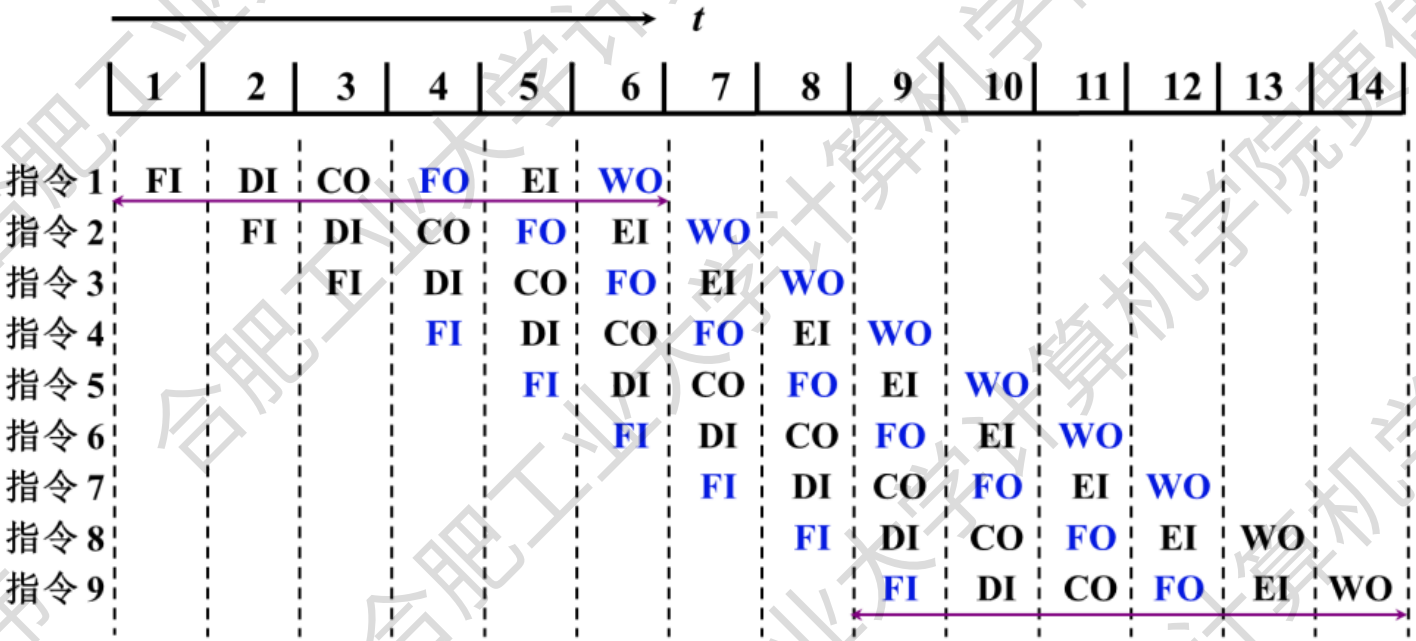
流水线中的相关问题

工厂里的流水线不能连续工作，可能是原料不到位，流水线上有员工不到位等原因。同理计算机CPU内部也有不能连续工作的情况，叫流水线阻塞。
流水线不能连续工作原因可能有：

①编译形成的程序不能发挥流水线的作用。
②存储器供应不上为连续流动所需的指令和数据。
③控制相关，主要是指遇到了程序转移指令(或中断）时，流水线不能继续处理后继指令。
④结构相关，又称资源相关，是指多条指令进入流水线后在同一机器时钟周期内争用同一个功能部件所发生的冲突。
⑤数据相关，即前一条指令执行过程中的数据结果是后一条指令执行过程中所需数据。由于时间差，后一条指令执行过程中，取不了这个数据。

三、影响指令流水线性能的因素 8.3

1. 结构相关　不同指令争用同一功能部件产生资源冲突



解决办法

指令 1 与指令 4 冲突　　指令1、指令3、指令 6 冲突

指令 2 与指令 5 冲突　　…

- 停顿
- 指令存储器和数据存储器分开
- 指令预取技术（适用于访存周期短的情况）

指令流水

8.3

【因素1】 结构相关

如 指令a使用运算器计算下一条指令的地址；指令b使用运算器完成指令运算；这两条指令就会发生资源冲突；

上图蓝色标识的时钟周期就会发生冲突；

如时钟周期4， 指令1的FO是访存取操作数；指令4的FI是访存取指令，这就会产生冲突； 这些冲突就会降低流水线性能；

解决方法：

- 1， 停顿；第4条指令的FI不在时钟周期4取指令，而停顿到时钟周期5取指令；
- 2， 指令存储器和数据存储器分开；
- 3， 指令预取技术（一次取多条指令，而不是每执行一条取一条指令）；

合肥工业大学计算机学院贾伟

2. 数据相关 8.3

不同指令因重叠操作，可能改变操作数的 读/写 访问顺序

- 写后读相关（RAW）

SUB R_1，R_2，R_3 ；（R_2）－（R_3）→ R_1

ADD R_4，R_5，R_1 ；（R_5）＋（R_1）→ R_4

- 读后写相关（WAR）

STA M，R_2 ；（R_2）→ M 存储单元

ADD R_2，R_4，R_5 ；（R_4）＋（R_5）→ R_2

- 写后写相关（WAW）

MUL R_3，R_2，R_1 ；（R_2）×（R_1）→ R_3

SUB R_3，R_4，R_5 ；（R_4）－（R_5）→ R_3

解决办法 • 后推法 • 采用 旁路技术

指令流水

8.3

不同指令重叠操作，流水线分为多个段，可能多个读段，或多个写段，通常情况下前段是读，后段是写，可能改变操作数的读写顺序；
（如 指令1 先读后写； 指令2 先读后写；）

- 写后读相关；先写后读相关；
- 读后写相关；
- 写后写相关；

- 解决方法：
- **后推法：**以写后读为例，等待写入R1成功后，然后再读R1的数据；
- **采用旁路技术：**以写后读为例，不需要等待指令1的结果写入R1，只要指令1的结果一旦在运算器的输出端生成以后，直接把结果送入到指令2的运算器的输入端； 这样会更好节约时间；

合肥工业大学计算机学院贾伟

3. 控制相关 8.3

由转移指令引起

```
      LDA   # 0
      LDX   # 0
M     ADD   X, D
      INX
      CPX   # N
      BNE   M
      DIV   # N
      STA   ANS
```

BNE 指令必须等

CPX 指令的结果

才能判断出

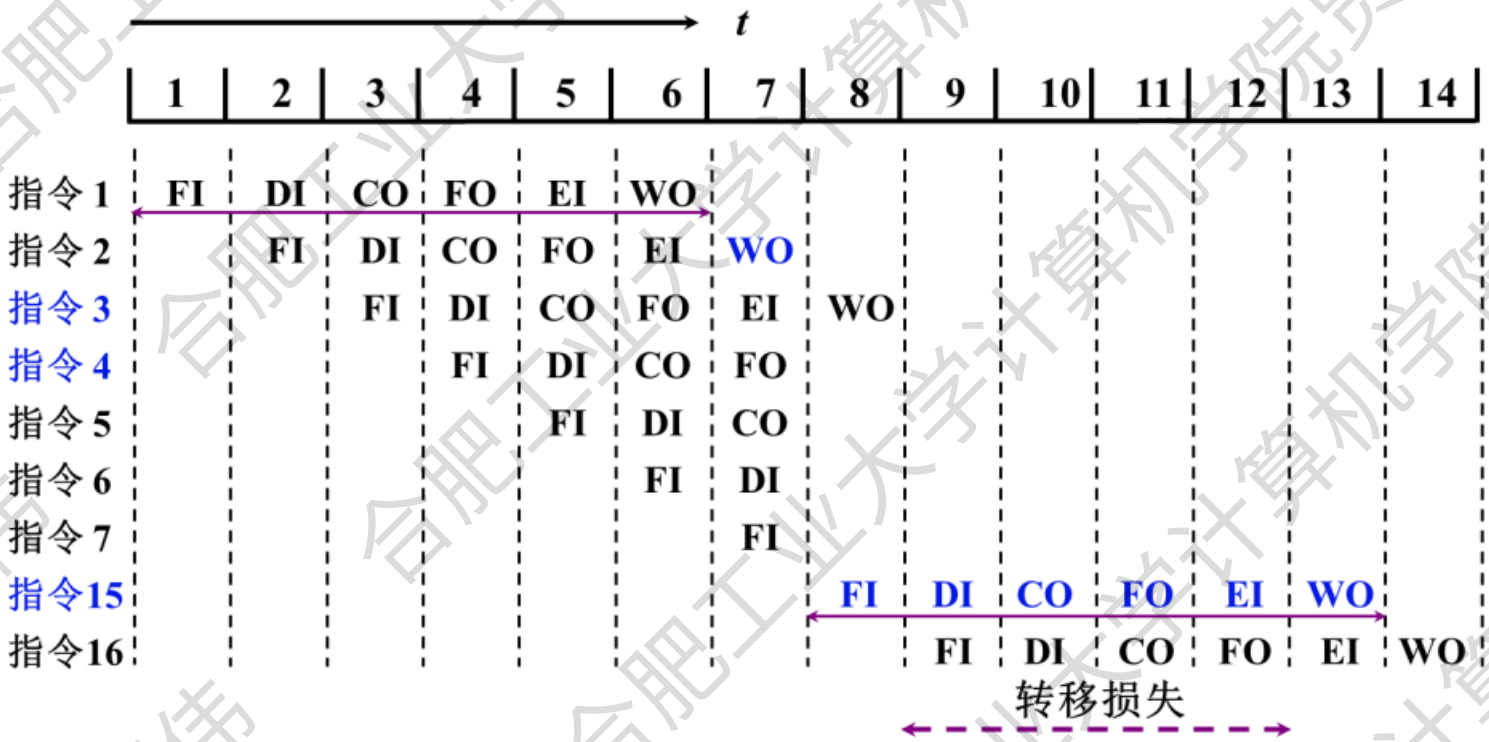
是转移

还是顺序执行

3. 控制相关

8.3

设 指令3 是转移指令



四、流水线性能

8.3

1. 吞吐率

单位时间内 流水线所完成指令 或 输出结果 的 数量

设 m 段的流水线各段时间为 Δt

- 最大吞吐率 (m=1时)

$$T_{pmax} = \frac{1}{\Delta t}$$

- 实际吞吐率 当未考虑相关问题时，连续处理n条指令的实际吞吐率为

连续处理 n 条指令的吞吐率为

$$T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$$

2. 加速比 S_p

8.3

m 段的 流水线的速度 与等功能的 非流水线的速度 之比

设流水线各段时间为 Δt

完成 n 条指令在 m 段流水线上共需

$$T = m \cdot \Delta t + (n-1)\Delta t$$

完成 n 条指令在等效的非流水线上共需

$$T' = nm \cdot \Delta t$$

则

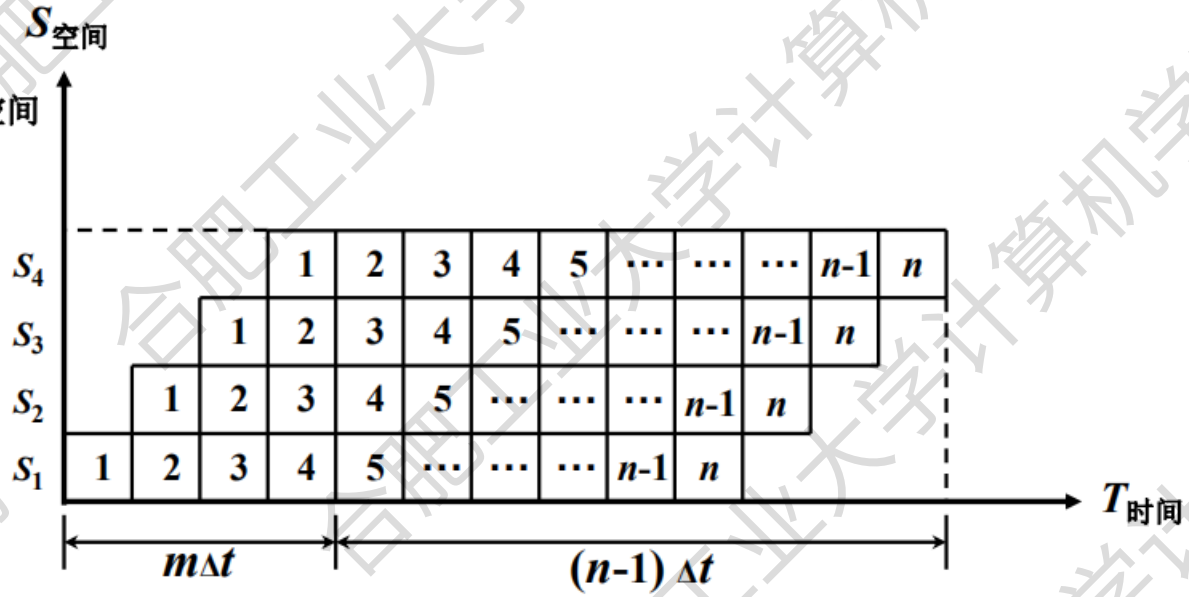
$$S_p = \frac{nm \cdot \Delta t}{m\Delta t + (n-1)\Delta t} = \frac{nm}{m+n-1}$$

3. 效率

8.3

流水线中各功能段的 利用率

由于流水线有 建立时间 和 排空时间
因此各功能段的 设备不可能 一直 处于 工作 状态



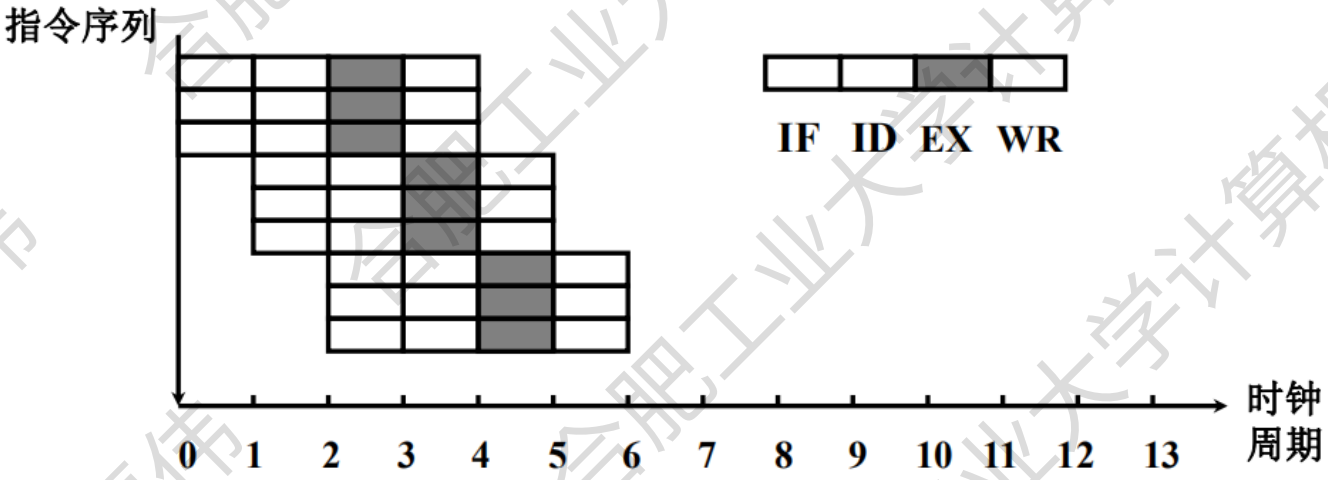
五、流水线的多发技术

8.3

1. 超标量技术

- 每个时钟周期内可 并发多条独立指令
  配置多个功能部件

- 不能调整 指令的 执行顺序
  通过编译优化技术，把可并行执行的指令搭配起来



2. 超流水线技术

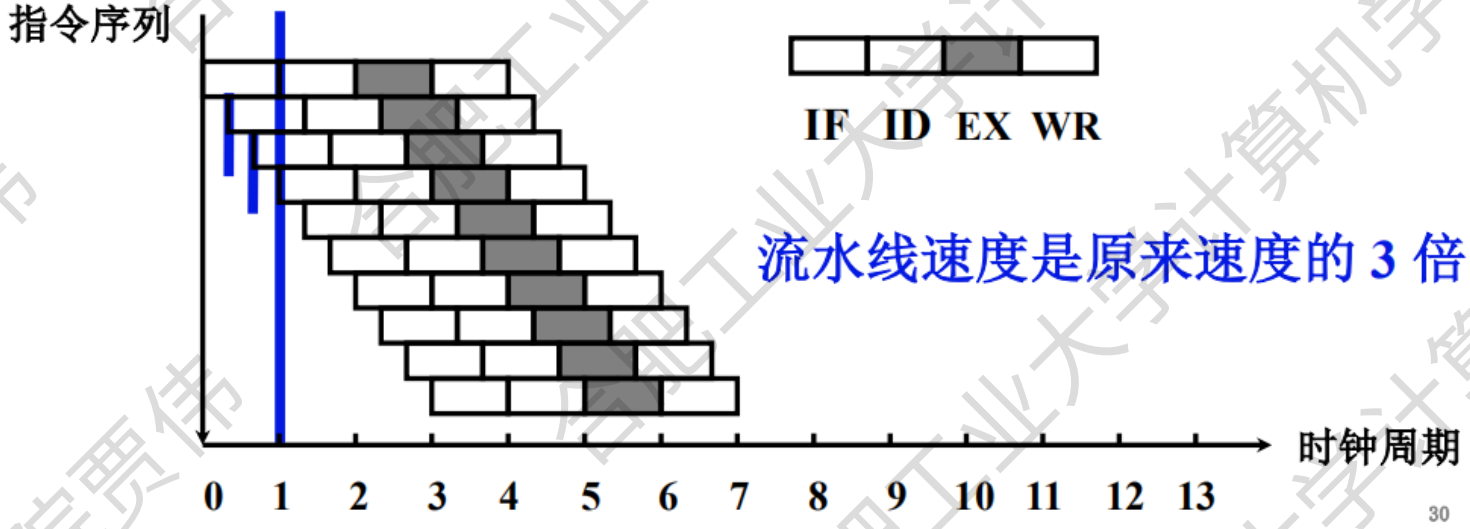
8.3

- 在 一个时钟周期 内 再分段 （3 段）

  在一个时钟周期内 一个功能部件使用多次（3 次）

- 不能调整 指令的 执行顺序

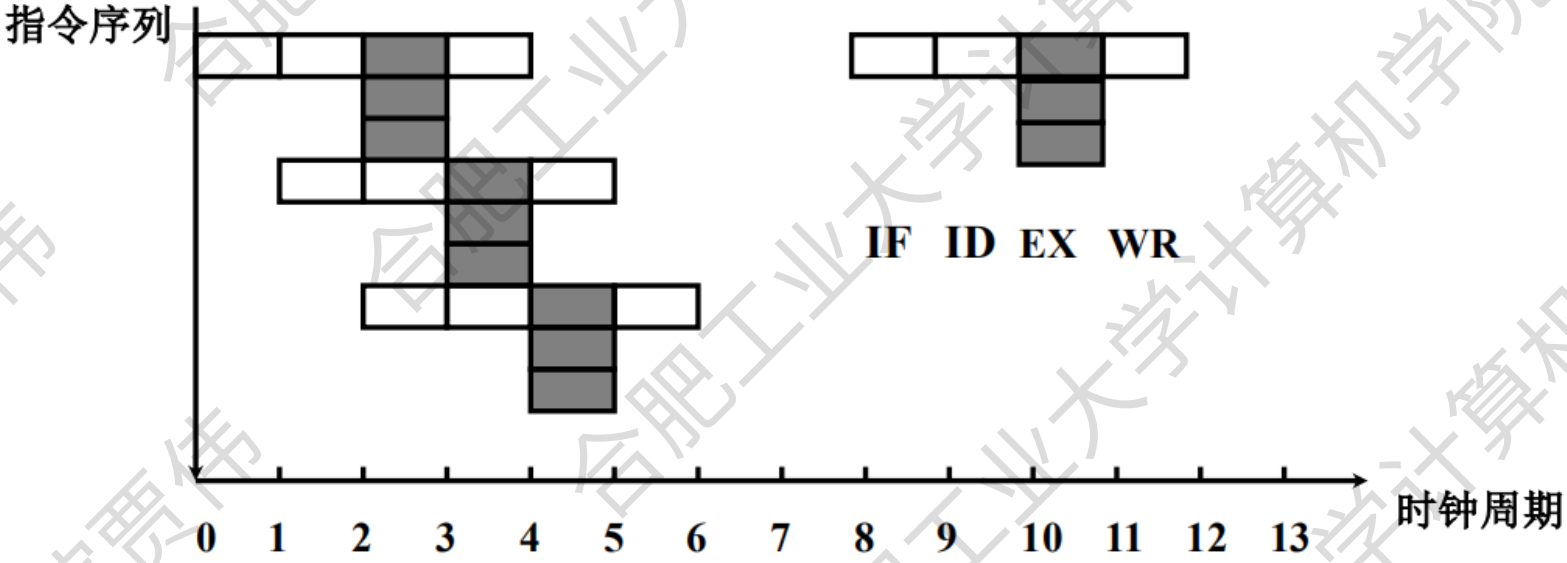
  靠编译程序解决优化问题



3. 超长指令字技术

8.3

- 由编译程序 挖掘 出指令间 潜在 的 并行性，
  将 多条 能 并行操作 的指令组合成 一条
  具有 多个操作码字段 的 超长指令字（可达几百位）
- 采用 多个处理部件

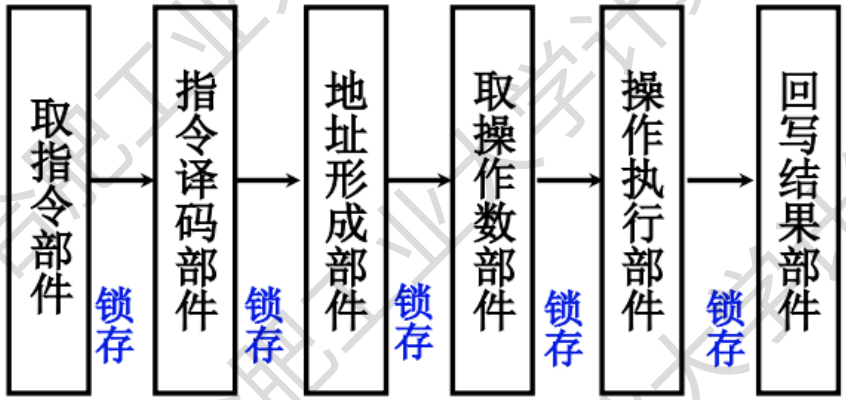


六、流水线结构

8.3

1. 指令流水线结构

完成一条指令分 6 段， 每段需一个时钟周期



若 流水线不出现断流　　1 个时钟周期出 1 结果

不采用流水技术　　6 个时钟周期出 1 结果

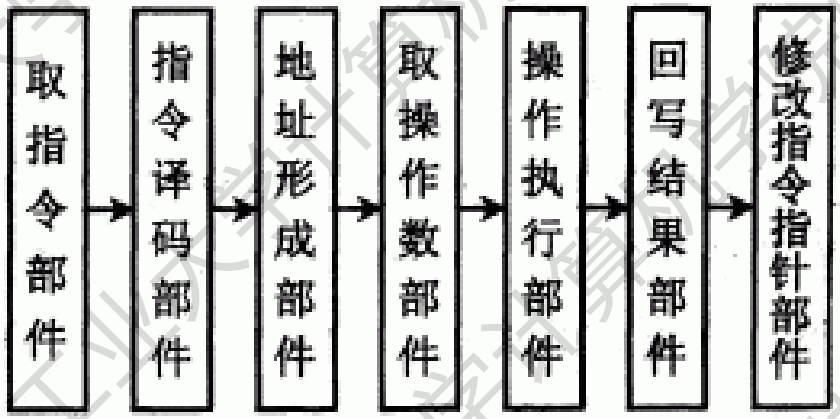
理想情况下，6 级流水 的速度是不采用流水技术的6 倍

指令流水

8.3

1．指令流水线结构

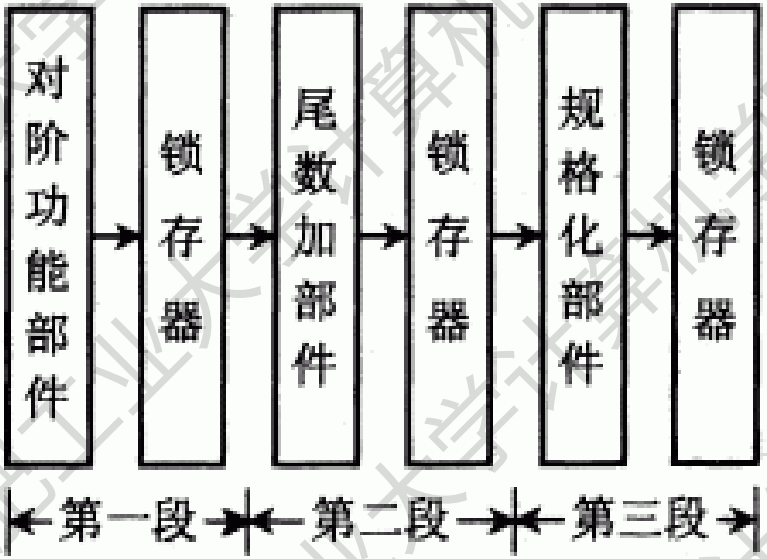
指令流水线是将指令的整个执行过程用流水线进行分段处理，典型的指令执行过程分为取指令—指令译码—形成地址—取操作数—执行指令—回写结果—修改指令指针这几阶段，与此相对应的指令流水线结构由下图所示的几个部件组成。



指令流水线对机器性能的改善程度取决于把处理过程分解成多少个相等的时间段数。如上述共分7段， 若每一段需一个时钟周期，则当不采用流水技术时，需7个时钟周期出一个结果。 采用流水线后，假设流水线不出现断流(如遇到转移指令)， 则除第一条指令需7个时钟周期出结果外，以后所有的指令都是一个时钟周期出一个结果。因此，在理想的情况下（流水线不断流）该流水线的速度约提高到7倍。

2．运算流水线

上述讨论的指令流水线是指令级的流水技术，实际上流水技术还可用于部件级。如浮点加法运算，可以分成“对阶”“尾数加”及“结果规格化”三段，每一段都有一个专门的逻辑电路完成操作，并将其结果保存在锁存器中，作为下一段的输入。如下图所示，当对阶完成后，将结果存入锁存器，便又可进入下一条指令的对阶运算。

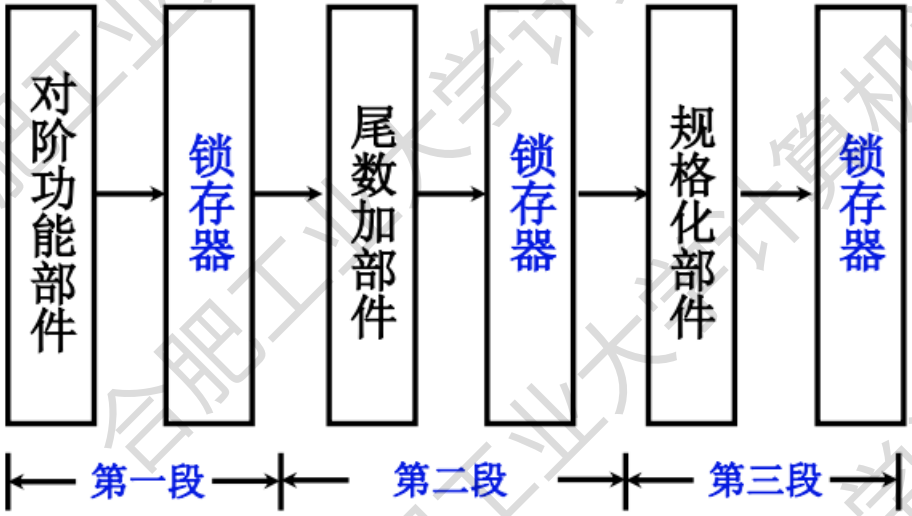


若执行浮点乘运算也按浮点加运算那样分段，即分成阶码运算、尾数乘和结果规格化三级流水线，就不够合理。因为尾数乘所需的时间比阶码运算和规格化操作长得多，而且尾数乘可以和阶码运算同时进行，因此，尾数乘本身就可以用流水线。

2. 运算流水线

完成 浮点加减 运算 可分

对阶、尾数求和、规格化 三段



分段原则　每段 操作时间 尽量 一致

由上图可见，流水线相邻两段在执行不同的操作，因此在相邻两段之间必须设置锁存器或寄存器，以保证在一个周期内流水线的输入信号不变。这一指导思想也适用于指令流水。此外，只有当流水线各段工作饱满时，才能发挥最大作用。上例中如果浮点运算没有足够的数据来源，那么流水线中的某些段、甚至全部段都处于空闲状态，使流水线的作用没有充分发挥。因此具体是否采用流水线技术，以及在计算机的哪一部分采用流水线技术需根据情况而定。

合肥工业大学计算机学院贾伟

8.4 中断系统

一、中断的概念

- 中断：CPU在执行程序的过程中，当出现异常情况或特殊请求处理时，CPU暂停当前正在执行的程序，转而去执行更为紧急的中断服务程序，并能在中断服务程序处理结束后，自动返回被暂停的程序继续执行。
- 中断系统：用于实现中断的软件和硬件组成的系统
- 一般适用于随机出现的服务，且一旦提出要求应立即实现。这种方式节省了CPU的时间，但硬件结构对复杂一些。

8.4 中断系统

中断

- 中断是指CPU对I/O设备发来的中断信号的一种响应
- 中断装置和中断处理程序统称为中断系统。中断系统是计算机的重要组成部分。实时控制、故障自动处理、计算机与外围设备间的数据传送往往采用中断系统。中断系统的应用大大提高了计算机效率。

合肥工业大学计算机学院贾伟

8.4 中断系统

计算机的中断系统能够加强CPU对多任务事件的处理能力。**中断机制是现代计算机系统中的基础设施之一**，它在系统中起着通信网络作用，以协调系统对各种外部事件的响应和处理。中断是实现多道程序设计的必要条件。中断是CPU对系统发生的某个事件作出的一种反应。引起中断的事件称为**中断源**。中断源向CPU提出处理的请求称为**中断请求**。发生中断时被打断程序的暂停点成为**断点**。CPU暂停现行程序而转为响应中断请求的过程称为**中断响应**。处理中断源的程序称为**中断处理程序**。CPU执行有关的中断处理程序称为**中断处理**。而返回断点的过程称为**中断返回**。中断的实现由实行软件和硬件综合完成，硬件部分叫做**硬件装置**，软件部分称为**软件处理程序**。

8.4 中断系统

1. 中断流程

中断请求-中断响应-保护断点-中断服务-中断返回。

2. 中断概念

（1）**中断系统：**实现中断功能的硬件系统和软件系统

（2）**中断源：**产生中断的请求源

（3）**中断请求：**中断源向CPU提出的处理请求。

（4）**中断响应：**CPU暂时中止自身的事情，转去处理事件的过程

（5）**中断服务：**对事件的整个处理过程。也称为中断处理

（6）**中断返回：**中断处理完毕，在返回到原来被中止的地方

合肥工业大学计算机学院贾伟

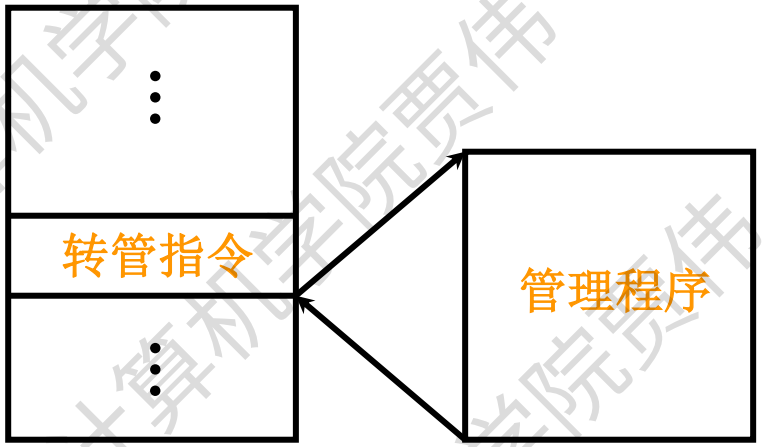
8.4 中断系统

一、概述

1. 引起中断的各种因素

(1) 人为设置的中断

如　转管指令



(2) 程序性事故　溢出、操作码不能识别、除法非法

(3)　硬件故障

(4)　I/O 设备

(5)　外部事件　　用 键盘中断 现行程序

中断的分类

1. 按中断来源分：

（1）内中断：发生在主机内部的中断称为内中断

（2）外中断：由主机外部事件引起的中断称为外中断

- 中断是外部设备引起的，又称外中断
- 陷入是由CPU内部事件所引起的中断，称为内中断或陷入
- 主要区别是信号的来源，即是来自CPU外部还是CPU内部

合肥工业大学计算机学院贾伟

2. 按中断服务程序入口地址的获取方式分

（1）向量中断：外部设备在提出中断请求的同时，通过硬件自动形成中断服务程序入口地址。CPU响应中断后，将根据向量地址直接转入相应中断服务程序。这种具有产生向量地址的中断称为向量中断。

（2）非向量中断：非向量中断在产生中断的同时不能直接提供中断服务程序入口地址，而只产生一个中断查询程序的入口地址。需要通过中断查询程序确定中断源和中断服务程序的入口地址。

3. 按中断源位置分：
   - 硬件中断
   - 软件中断
4. 按是否可屏蔽分：CPU根据需要可以禁止响应某些中断源的中断请求。
   - 可屏蔽中断：CPU可以禁止响应的中断
   - 不可屏蔽中断：CPU必须响应的中断。

合肥工业大学计算机学院贾伟

2. 中断系统需解决的问题

(1) 各中断源 如何 向 CPU 提出请求？

(2) 各中断源 同时 提出 请求 怎么办？

(3) CPU 什么 条件、什么 时间、以什么 方式 响应中断？

(4) 如何 保护现场？

(5) 如何 寻找入口地址？

(6) 如何 恢复现场，如何 返回？

(7) 处理中断的过程中又 出现新的中断 怎么办？

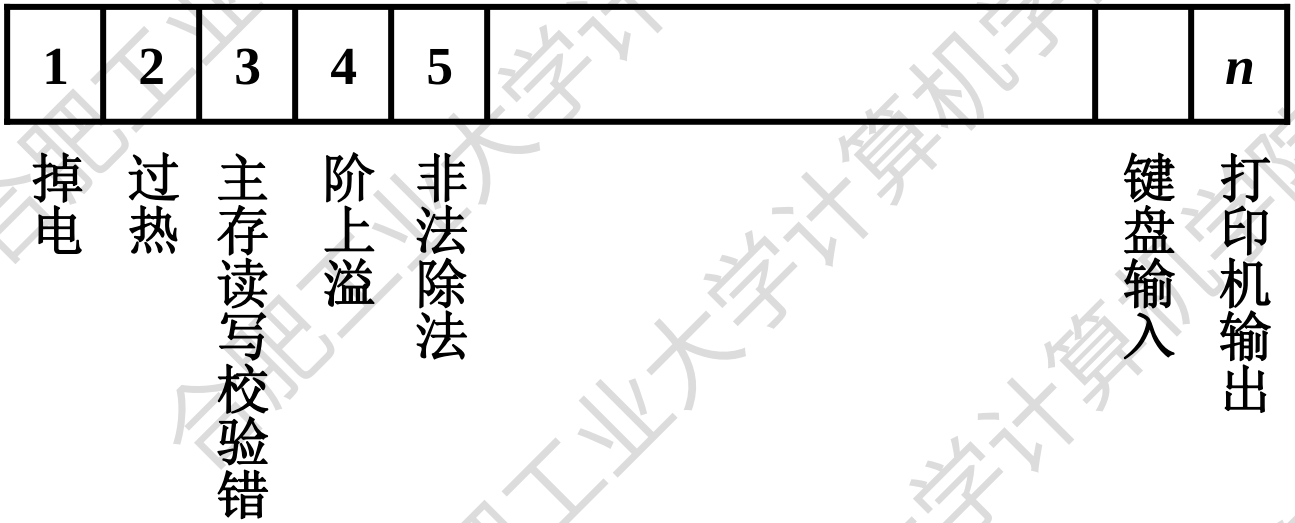
硬件 + 软件

二、中断请求标记和中断判优逻辑 8.4

1. 中断请求标记 INTR

一个请求源 一个 INTR 中断请求触发器 有中断，状态：1

多个INTR 组成 中断请求标记寄存器



中断请求触发器越多，说明计算机处理中断的能力越强

INTR 分散 在各个中断源的 接口电路中

INTR 集中 在 CPU 的中断系统 内

2. 中断判优逻辑

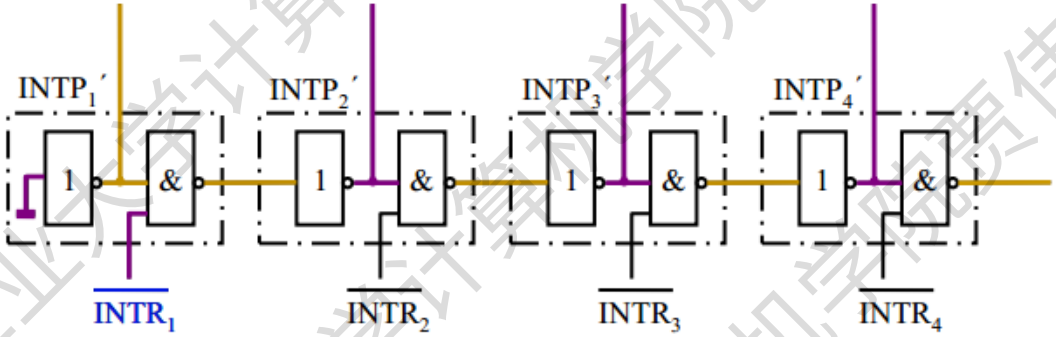
任何一个中断系统，在任一时刻，只能响应一个中断源的请求。但许多中断源提出请求都是随机的，当某一时刻有多个中断源提出中断请求时，中断系统必须按其优先顺序予以响应，这就叫中断判优。各中断源的优先顺序是根据该中断源若得不到及时响应，致使机器工作出错的严重程度而定的。例如，电源掉电对机器工作影响程度最大，优先等级为最高。又如“定点溢出”对机器正常工作影响也很大，若不及时响应，将使机器一切运行均无效，故它的优先等级也较高。对于I/O设备，则可按其速度高低安排优先等级，速度高的设备其优先级比速度低的设备高。

中断判优可用硬件实现，也可用软件实现。

2. 中断判优逻辑

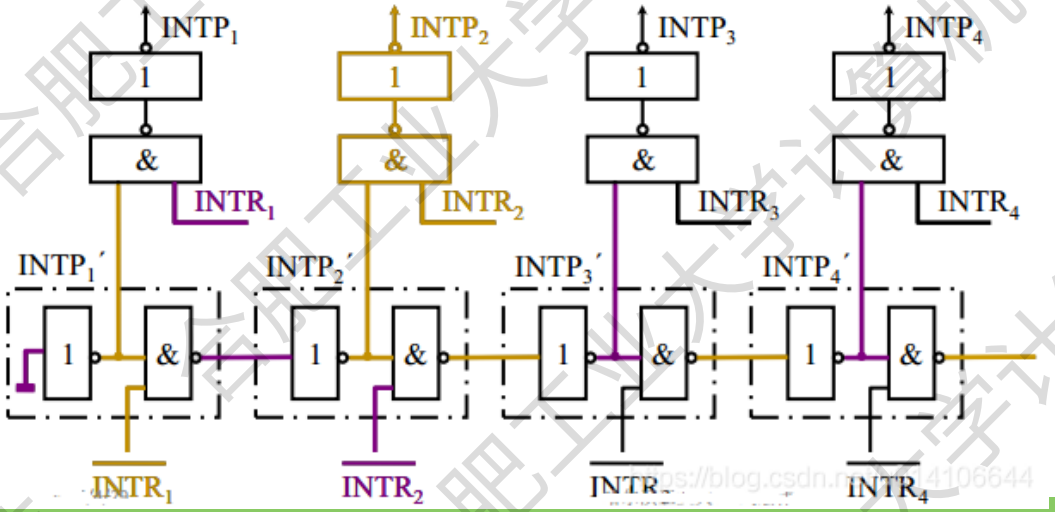
(1) 硬件实现（排队器）

① 分散 在各个中断源的 接口电路中　链式排队器



设备 1#、2#、3#、4# 优先级按 降序排列

$INTR_i = 1$ 有请求　　即 $\overline{INTR_i} = 0$

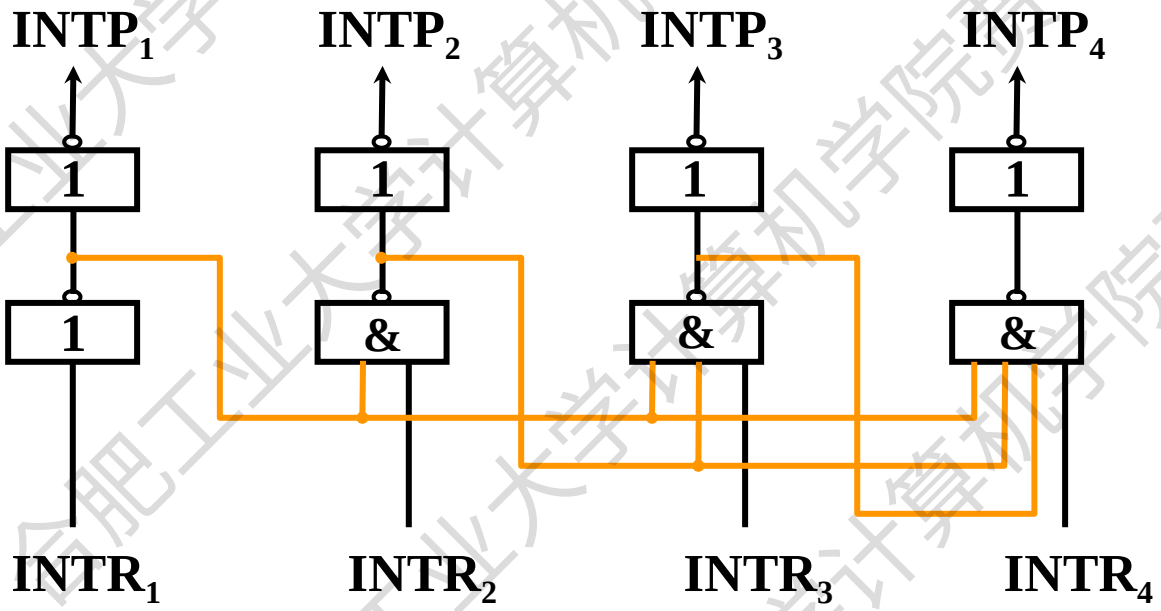


2. 中断判优逻辑

8.4

(1) 硬件实现（排队器）

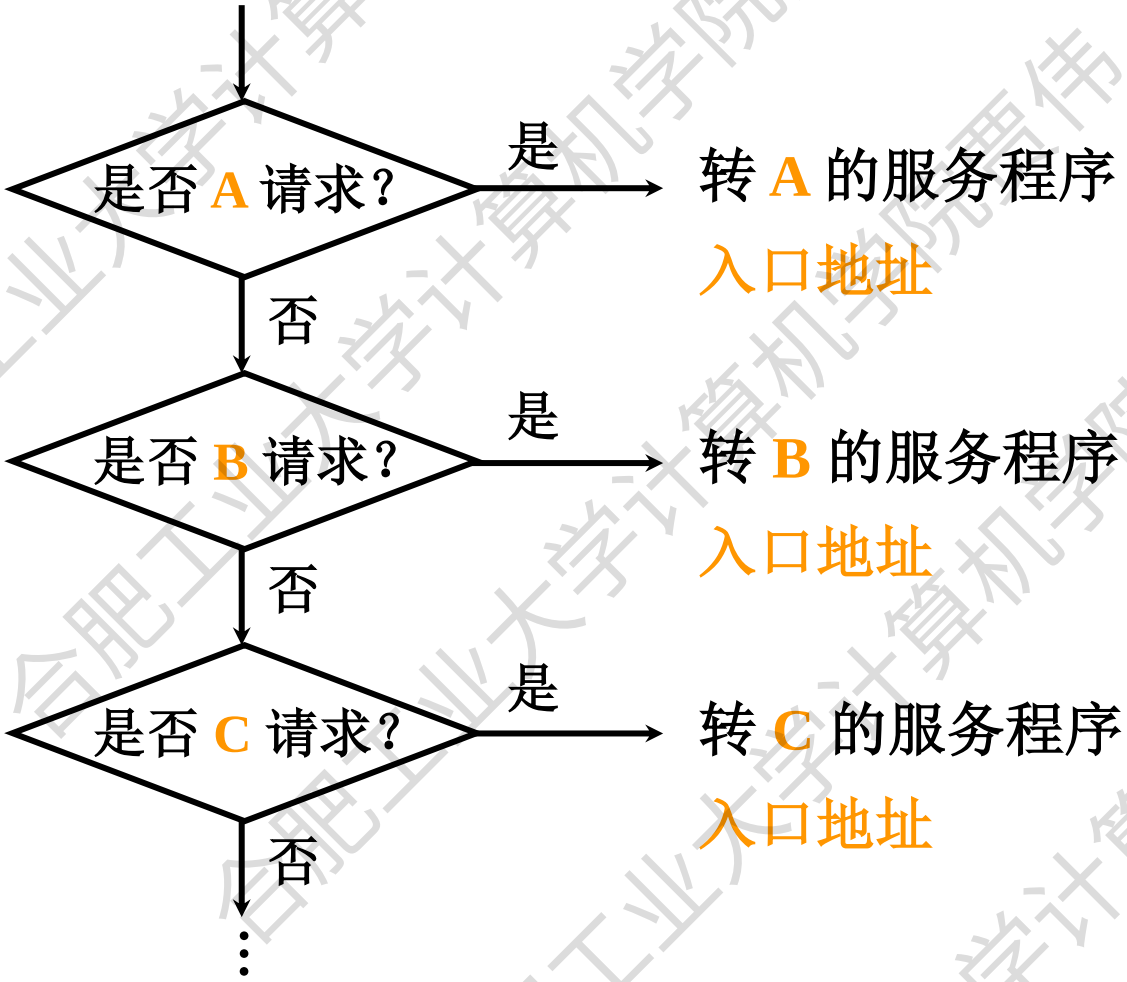
② 集中 在 CPU 内



$INTR_1$、$INTR_2$、$INTR_3$、$INTR_4$ 优先级 按 降序 排列

(2) 软件实现（程序查询）

A、B、C 优先级按 降序 排列

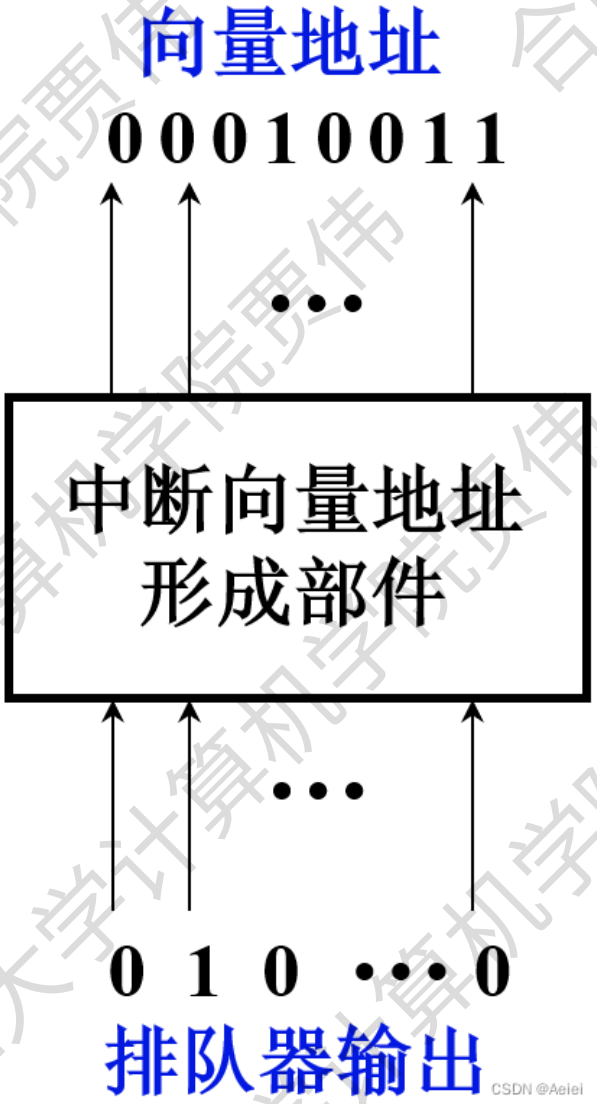


三、中断服务程序入口地址的寻找 8.4

1. 硬件向量法

CPU 一旦响应了 I/O 中断，就要暂停现行程序，转去执行该设备的中断服务程序。不同的设备有不同的中断服务程序，每个服务程序都有一个入口地址，CPU 必须找到这个人口地址。

所谓硬件向量入口地址的寻找也可用硬件或软件的方法来完成，这里只介绍硬件向量法，就是通过向量地址来寻找设备的中断服务程序入口地址，而且向量地址是由硬件电路产生的，如图所示。



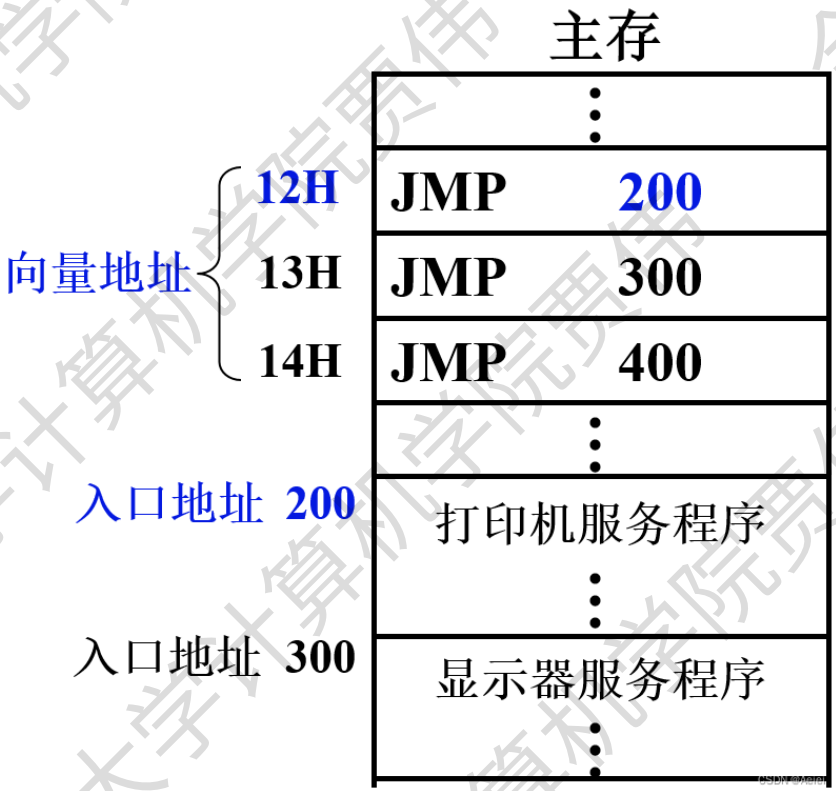
CSDN @Aelei

三、中断服务程序入口地址的寻找 8.4

1. 硬件向量法

中断向量地址形成部件的输入是来自排队器的输出 INTP1 _1，INTP2 _2，…，INTPn _n，它的输出是中断向量（二讲制代码表示），其位数与计算机可以处理中断源的个数有关，即一个中断源对应一个向量地址。可见，该部件实质上是一个编码器。

在I/O接口中的编码器又称为设备编码器。这里必须分清向量地址和中断服务程序的入口地址是两个不同的概念，右图是通过向量地址寻找入口地址的一种方案。其中 12H，13H，14H 是向量地址，200、300分别是打印机服务程序和显示器服务程序的入口地址。



三、中断服务程序入口地址的寻找 8.4

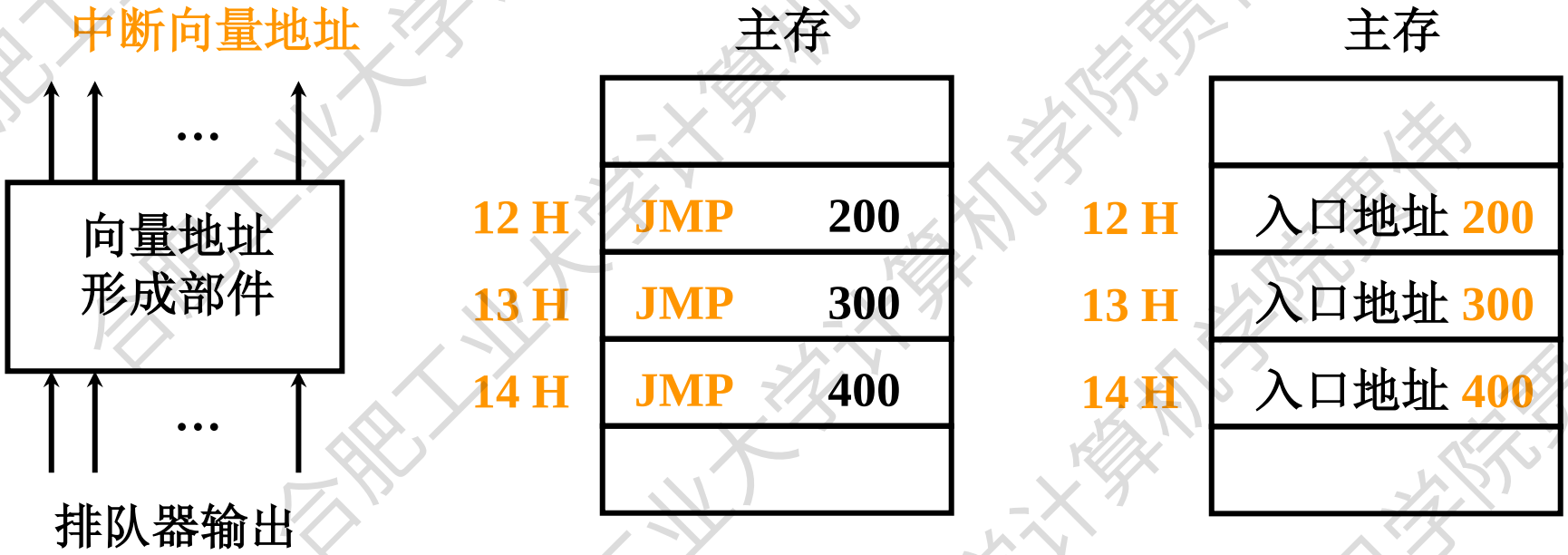
1. 硬件向量法

- 中断向量地址形成部件输入为排队器的输出，输出为中断向量，一个中断源对于一个向量地址，中断向量地址形成部件其实是一个编码器。
- 排队器输入，输出向量地址，在主存中向量地址为一条跳转指令，跳转到相应的中断服务程序入口地址
- 将排队器输出输入到中断向量地址形成部件（编码器）中，产生向量地址；向量地址所在的内存单元放置跳转指令指向具体的中断服务程序入口。或者内存单元直接存放中断服务程序的入口地址

合肥工业大学计算机学院贾伟

三、中断服务程序入口地址的寻找 8.4

1. 硬件向量法



向量地址　12H、13H、14H

入口地址　200、 300、 400

2. 软件查询法

八个中断源 1，2，…　8 按 降序 排列

中断识别程序（入口地址 M）

地 址	指 令	说 明
M	SKP　DZ $1^{\#}$	$1^{\#}$　D = 0 跳（D为完成触发器）
	JMP　$1^{\#}$ SR	$1^{\#}$　D = 1 转$1^{\#}$ 服务程序
	SKP　DZ $2^{\#}$	$2^{\#}$　D = 0 跳
	JMP　$2^{\#}$ SR	$2^{\#}$　D = 1 转$2^{\#}$ 服务程序
	⋮	
	SKP　DZ $8^{\#}$	$8^{\#}$　D = 0 跳
	JMP　$8^{\#}$ SR	$8^{\#}$　D = 1 转$8^{\#}$ 服务程序

四、中断响应

8.4

1. 响应中断的 条件

CPU响应I/O中断的条件是允许中断触发器必须为“1”，这一结论同样适合于其他中断源。在**中断系统中有一个允许中断触发器EINT，它可被开中断指令置“1”，也可被关中断指令置“0”。**当允许中断触发器为“1”时，意味着CPU允许响应中断源的请求；当其为“0”时，意味着CPU禁止响应中断。故当EINT=l，且有中断请求(即中断请求标记触发器INTR=1)时，CPU可以响应中断。

合肥工业大学计算机学院贾伟

2. 响应中断的 时间

与响应I/O中断一样，CPU总是在指令执行周期结束后，响应任何中断源的请求。在指令执行周期结束后，若有中断，CPU则进入中断周期；若无中断，则进入下一条指令的取指周期。

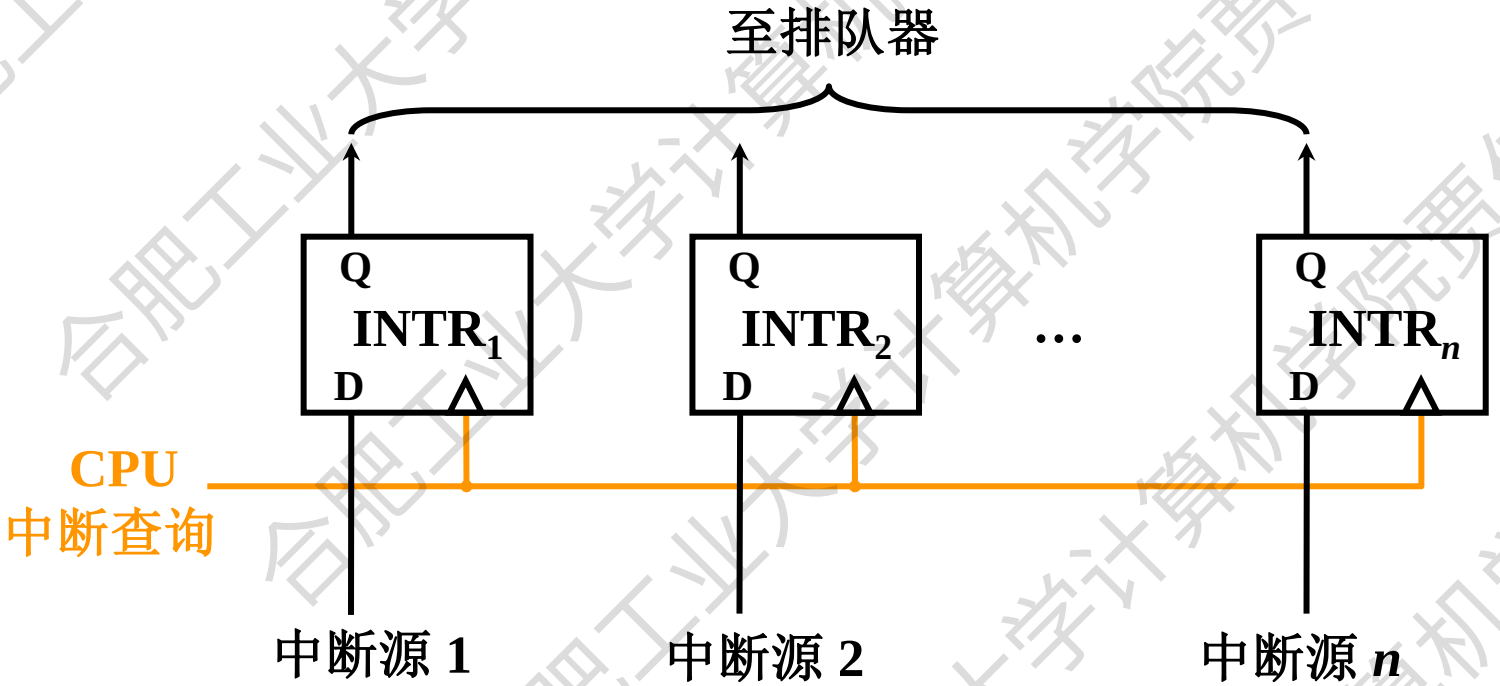
CPU之所以在指令仍执行周期后进入中断周期，是因为CPU在执行周期的结束时刻统一向所有中断源发中断查询信号，只有此时，CPU才能获知哪个中断源有请求。如图所示，图中$INTR_i$(i=1,2……)是各个中断源的中断请求触发器，触发器的数据端来自各中断源，当它们有请求时，数据端为“1”，而且只有当CPU查询信号到触发器的时钟端时，才能置“1” $INTR_i$。

在某些计算机中有些指令执行时间很长，若CPU的查询信号一律安排在执行周期结束时刻，有可能因CPU发现中断请求过迟而出差错。为此，可在指令执行过程中设置若干个查询断点，CPU在每个“查询断点”时刻均发中断查询信号，以便发现有中断请求便可及时响应。

四、中断响应

2. 响应中断的 时间

指令执行周期结束时刻由CPU 发查询信号



3. 中断隐指令

8.4

CPU响应中断后，即进入中断周期。在中断周期内，CPU要自动完成一系列操作，如：**(1)保护程序断点。**保护程序断点就是要将当前程序计数器PC的内容(程序断点)保存到存储器中。它可以存在存储器的特定单元(如0号地址)内，也可以存入堆栈。

(2)寻找中断服务程序的入口地址。由于中断周期结束后进入下条指令(即中断服务程序的第一条指令)的取指周期，因此在中断周期内必须设法找到中断服务程序的入口地址。由于入口地址有两种方法获得，因此在中断周期内也有两种方法寻找入口地址：

其一，在中断周期内，将向量地址送至PC(对应硬件向量法)，使CPU下一条执行无条件转移指令，转至中断服务程序的入口地址。

其二，在中断周期内，将软件查询入口地址的程序(又叫中断识别程序)其首地址送至PC，使CPU执行中断识别程序，找到入口地址(对应软件查询法)。

(3)关中断。CPU进入中断周期，意味着CPU响应了某个中断源的请求，为了确保CPU响应后所需作的一系列操作不至于又受到新的中断请求的干扰，在中断周期内必须自动关中断，以禁止CPU再次响应新的中断请求。图中允许中断触发器EINT和中断标记触发器INT可选用标准的R-S触发器。当进入中断周期时，则T为“1”状态，触发器原端输出有一个正跳变，经反相后产生一个负跳变，使EINT置0，即关中断。

3. 中断隐指令

8.4

(1) 保护程序断点

断点存于 特定地址（ 0 号地址）内　　断点 进栈

(2) 寻找服务程序入口地址

向量地址 ⟶ PC　（硬件向量法）

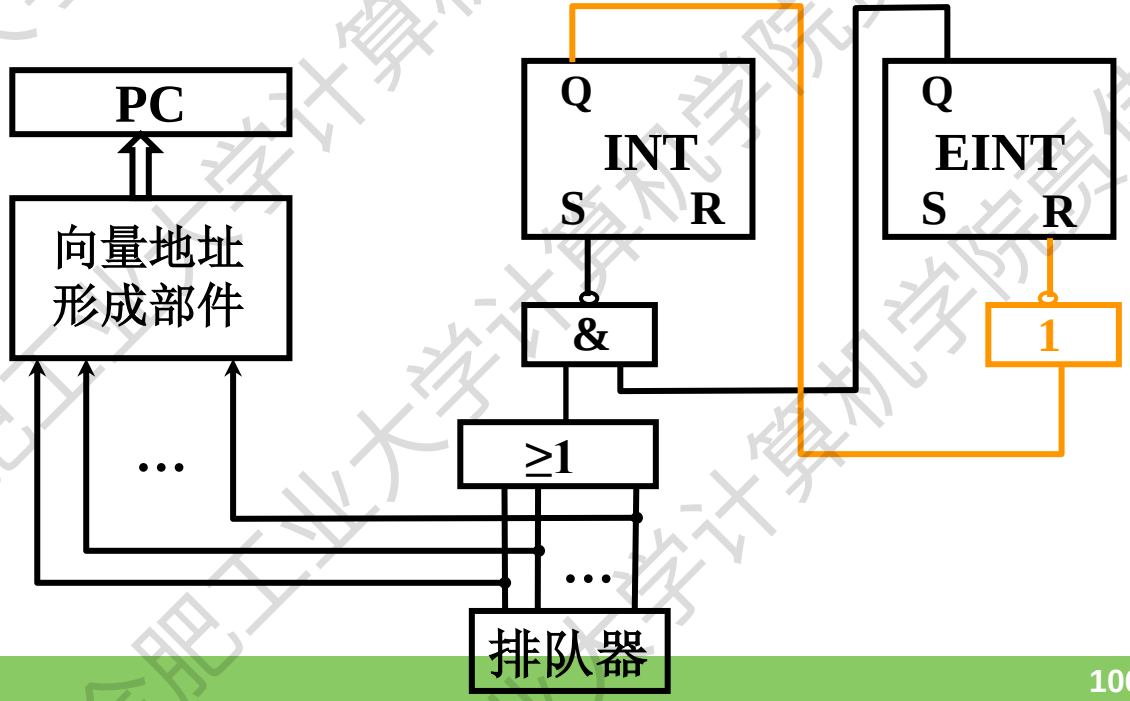
中断识别程序 入口地址　M ⟶ PC　（软件查询法）

(3) 硬件 关中断

INT 中断标记

EINT 允许中断

R – S 触发器



五、保护现场和恢复现场 8.4

保护现场应该包括保护程序断点和保护CPU内部各寄存器内容的现场两个方面。程序断点的现场由中断隐指令完成，各寄存器的内容可在中断服务程序中由用户(或系统)用机器指令编程实现。

恢复现场是指在中断返回前，必须将寄存器的内容恢复到中断处理前的状态，这部分工作也由中断服务程序完成。

合肥工业大学计算机学院贾伟

五、保护现场和恢复现场

8.4

1. 保护现场
   - 断点 —— 中断隐指令 完成
   - 寄存器 内容 —— 中断服务程序 完成
2. 恢复现场 —— 中断服务程序 完成

中断服务程序

保护现场	PUSH
其它服务程序	视不同请求源而定
恢复现场	POP
中断返回	IRET

六、中断屏蔽技术

8.4

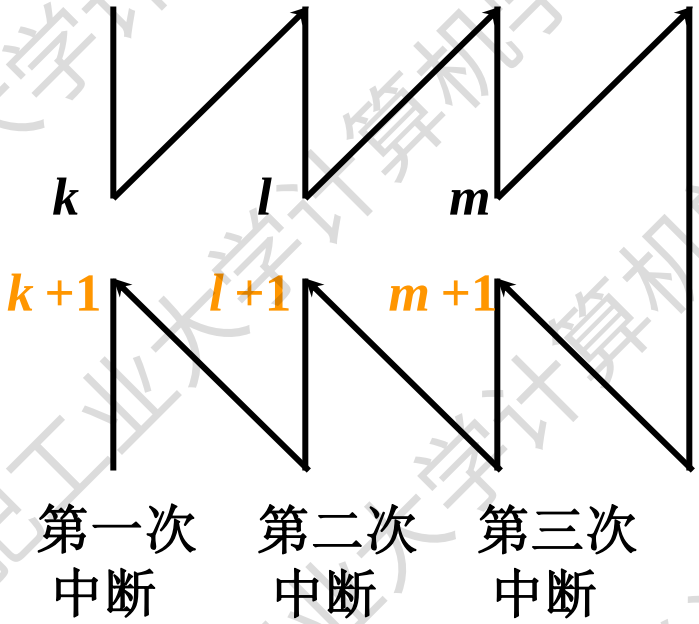
1. 多重中断的概念

当CPU正在执行某个中断服分程序时，另一个中断源又提出了新的中断请求，而CPU又响应了这新的请求，暂时停止了正在运行的服务程序，转去执行新的中断服务程序，这就叫多重中断，或叫中断嵌套。如果CPU对新的请求不予响应，待执行完当前的服务程序后再响应，即为单重中断。中断系统若要具有处理多重中断的功能，必须具备各项条件。

六、中断屏蔽技术

8.4

1. 多重中断的概念



程序断点 $k+1$, $l+1$, $m+1$

2. 实现多重中断的条件

(1) 提前 设置 开中断 指令

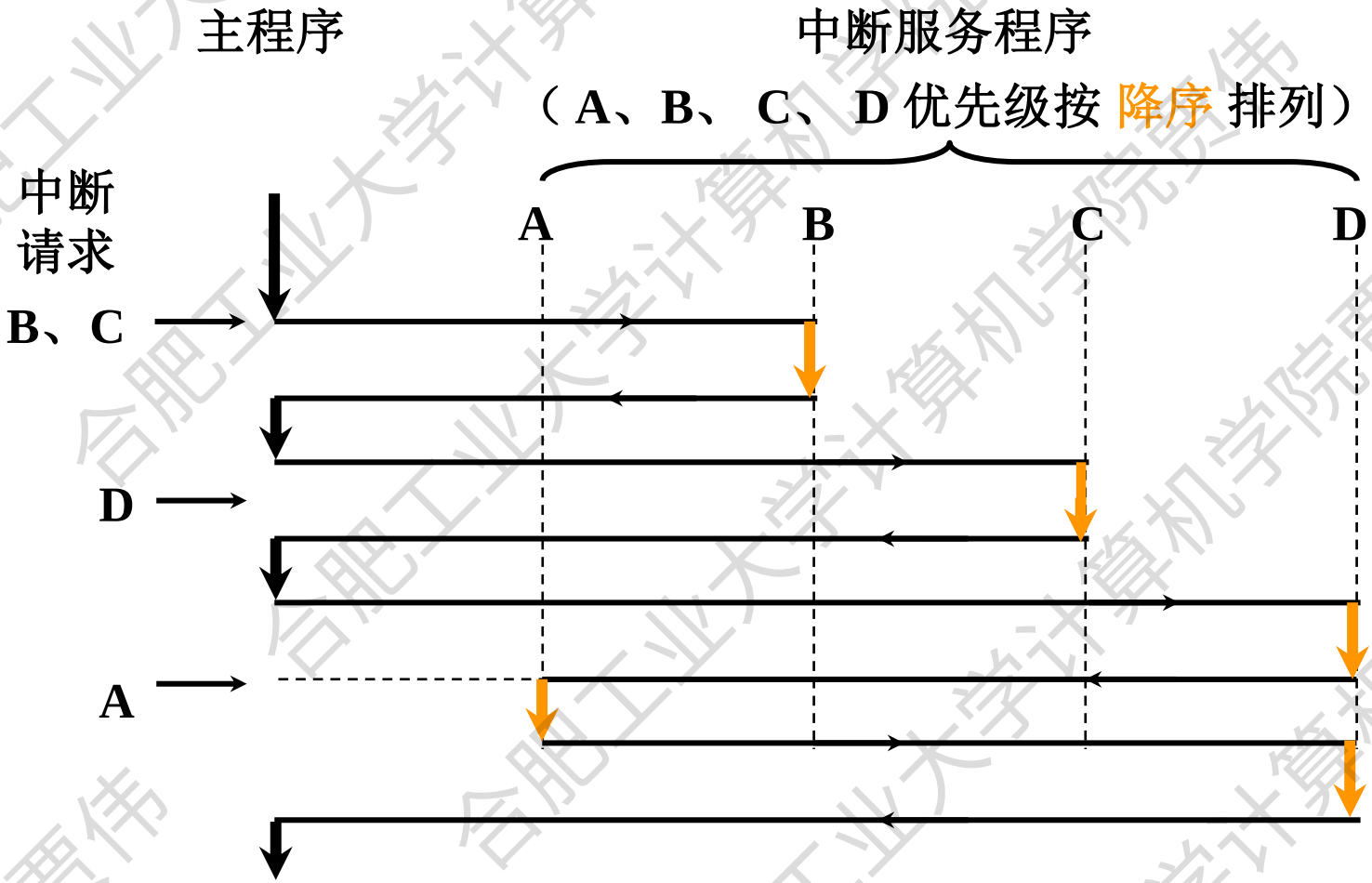
由上述分析可知，CPU进入中断周期后，由中断隐指令自动置“0”ENIT，即关中断，这就意味着CPU在执行中断服务程序中禁止响应新的中断请求。

CPU若想再次响应中断请求，必须开中断，这一任务通常由中断服务程序中的开中断指令实现。由于开中断指令设置的位置不同，决定了CPU能否实现多重中断。多重中断“开中断”指令的位置前于单重中断，从而保证了多重中断允许出现中断嵌套。

合肥工业大学计算机学院贾伟

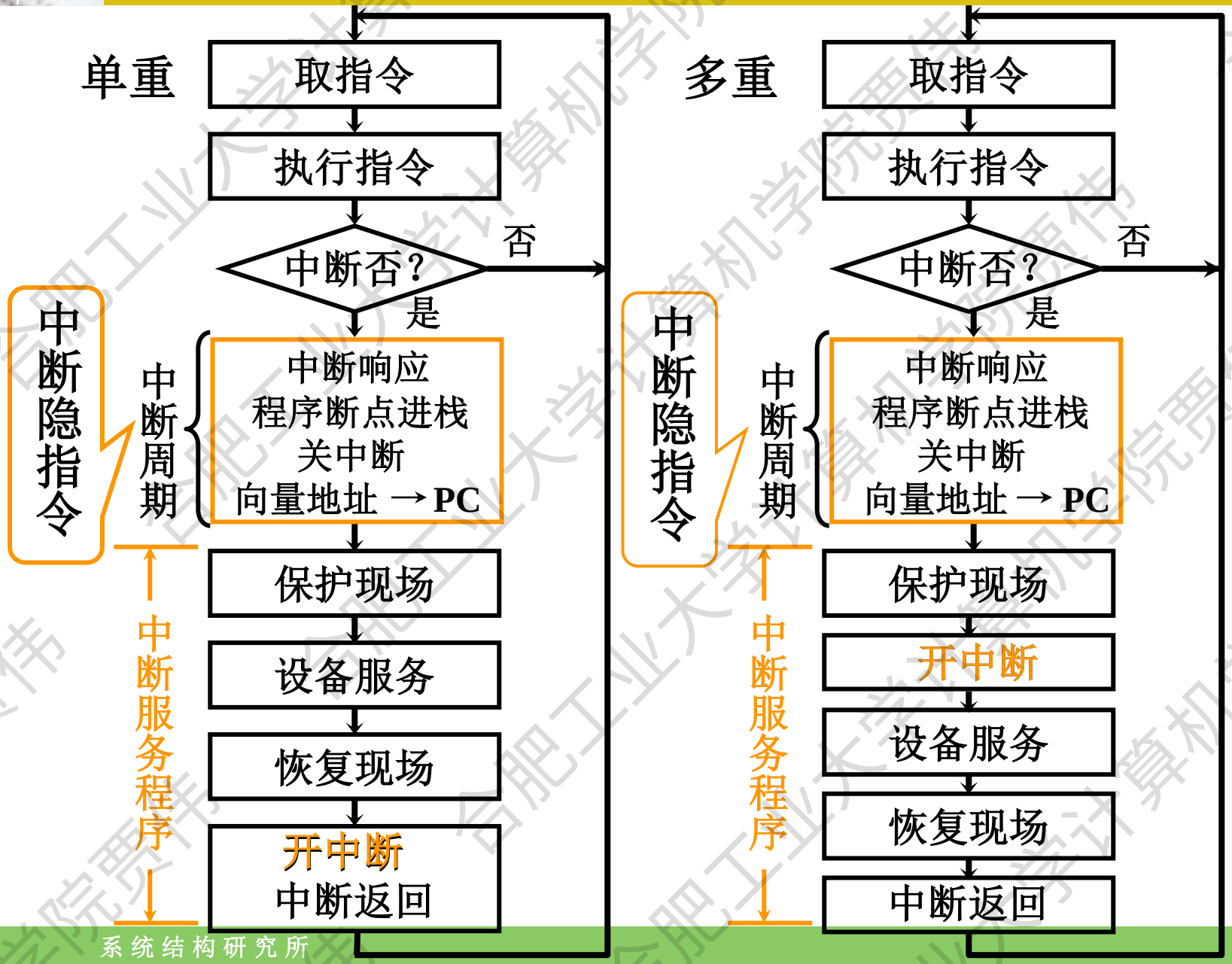
2. 实现多重中断的条件

(2) 优先级别高 的中断源 有权中断优先级别低 的中断源



单重中断和多重中断的服务程序流程

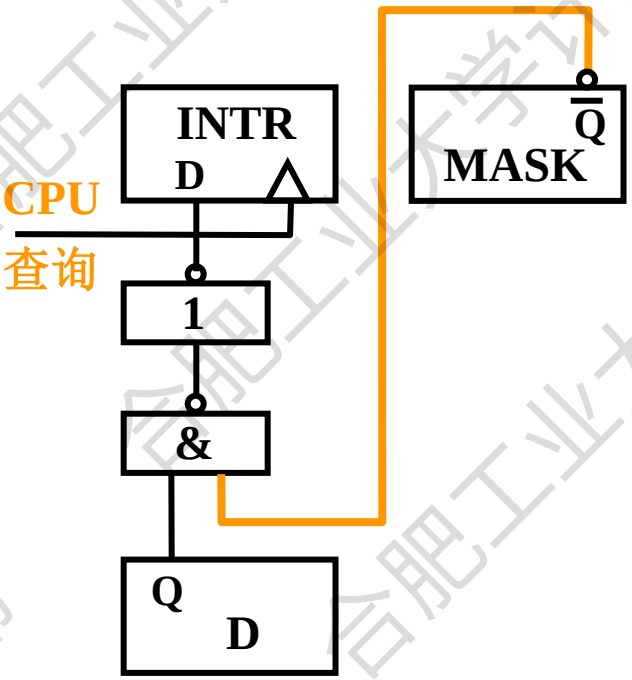
5.5



3. 屏蔽技术

8.4

(1) 屏蔽触发器的作用



MASK = 0（未屏蔽）

INTR 能被置 “1”

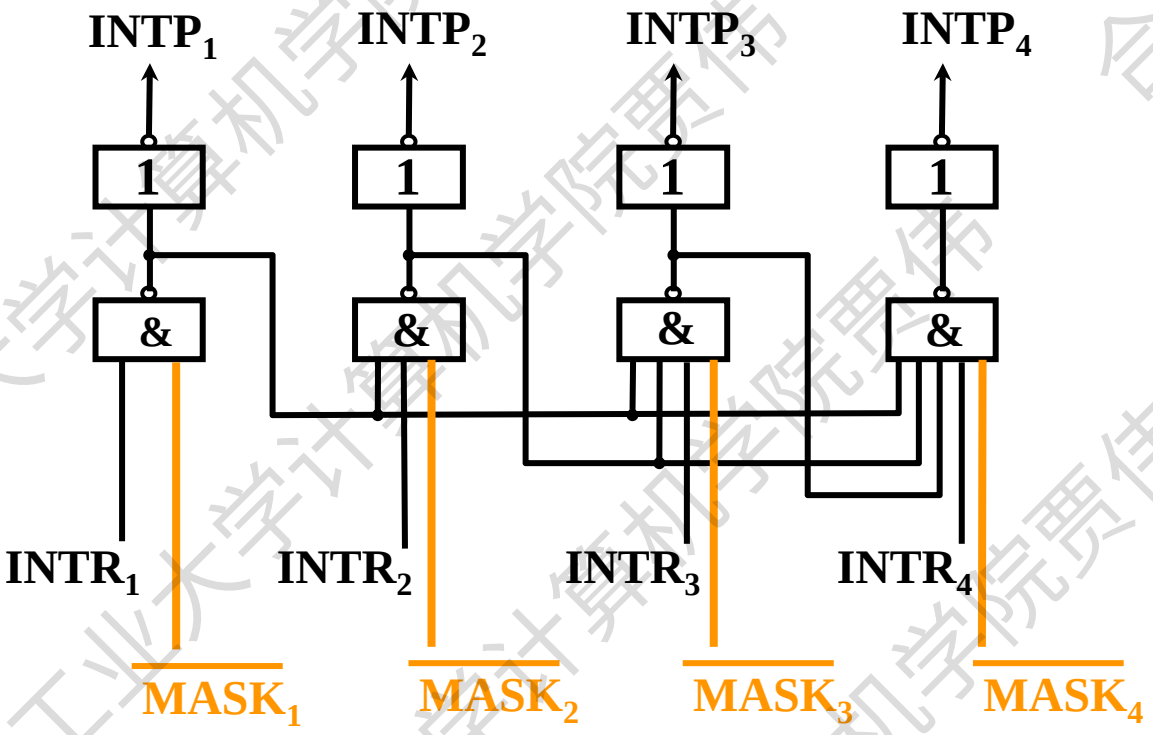
图示出了程序中断接口电路中完成触发器D、中断请求触发器INTR和屏蔽触发器MASK三者之间的关系。当该中断源被屏蔽时(MASK=1)，此时即使D＝1，中断查询信号到来时刻将INTR置0，CPU接收不到该中断源的中断请求，即它被屏蔽。若该中断源未被屏蔽(MASK=0)，则当设备工作已完成时(D＝1)，中断查询信号将INTR置1，表示该中断源向CPU发出中断请求，该信号送至排队器进行优先级判断。

3. 屏蔽技术

8.4

(1) 屏蔽触发器的作用

如果排队器集中设在CPU内，加上屏蔽条件，就可组成具有屏蔽功能的排队器，如图所示。



$MASK_i = 1$ （屏蔽）

$INTP_i = 0$ （不能被排队选中）

(2) 屏蔽字

显然，对应每个中断请求触发器就有一个屏蔽触发器，将所有屏蔽触发器组合在一起，便构成一个屏蔽寄存器，屏蔽寄存器的内容称作屏蔽字。屏蔽字与中断源的优先级别是一一对应的，上表是对应16个中断源的屏蔽字，每个屏蔽字由左向右排序为第1，2，3.....共16位。不难发现，每个中断源对应的屏蔽字是不同的。1级中断源的屏蔽字是16个1；2级中断源的屏蔽字是从第2位开始共15个1；3级中断源的屏蔽字是从第3位开始共14个1.....第16级中断源的屏蔽字只有第16位为1，其余各位为0。

在中断服务程序中设置适当的屏蔽字，能起到对优先级别不同的中断源的屏蔽作用。例如，1级中断源的请求已被CPU响应，若在其中断服务程序中(通常在开中断指令前)设置一个全“1”的屏蔽字，便可保证在执行1级中断服务程序过程中，CPU不再响应任何一个中断源(包括本级在内)的中断请求，即此刻不能实现多重中断。如果在4级中断源的服务程序中设置一个屏蔽字0001111111111111，由于第1～3位为0，意味着第1～3级的中断源未被屏蔽，因此在开中断指令后，比第4级中断源级别更高的1、2、3级中断源可以中断4级中断源的中断服务程序，实现多重中断。

(2) 屏蔽字

16个中断源 1，2，3，… 16 按 降序 排列

优先级	屏 蔽 字
1	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2	0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
3	0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1
4	0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1
5	0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1
6	0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1
⋮	⋮
15	0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1
16	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

(3) 屏蔽技术可改变处理优先等级

响应优先级　　不可改变

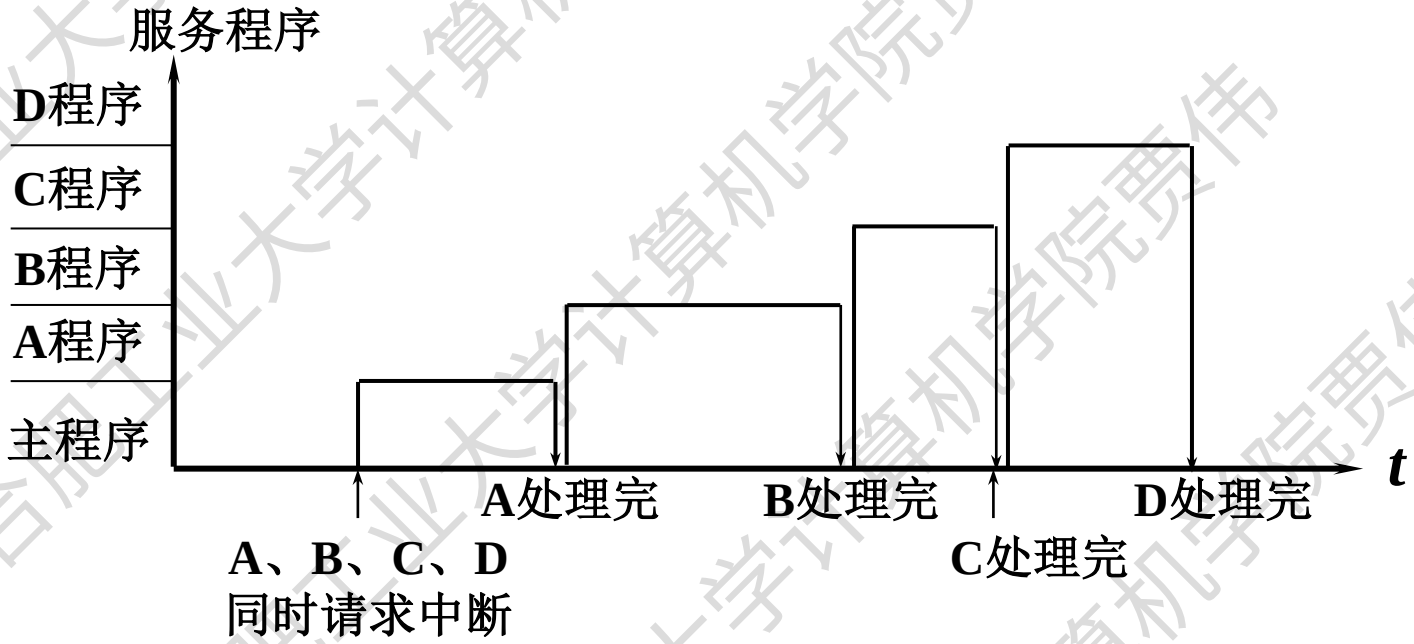
处理优先级　　可改变（通过重新设置屏蔽字）

中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级 A→B→C→D 降序排列

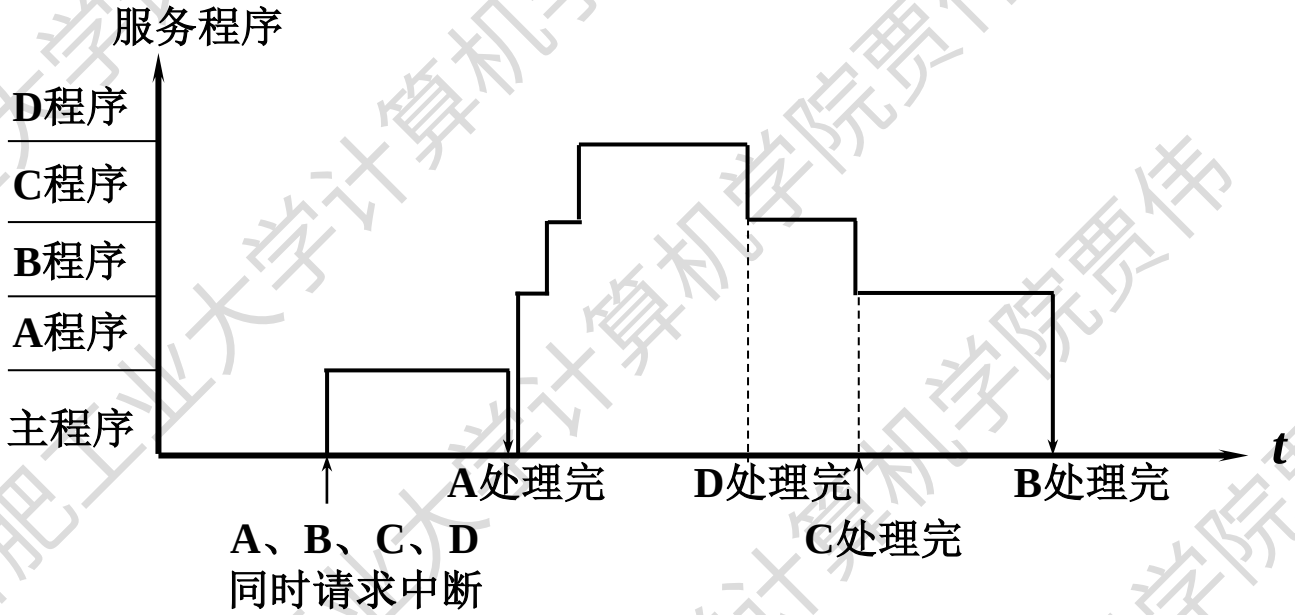
处理优先级 A→D→C→B 降序排列

(3) 屏蔽技术可改变处理优先等级



CPU 执行程序轨迹（原屏蔽字）

(3) 屏蔽技术可改变处理优先等级



CPU 执行程序轨迹（新屏蔽字）

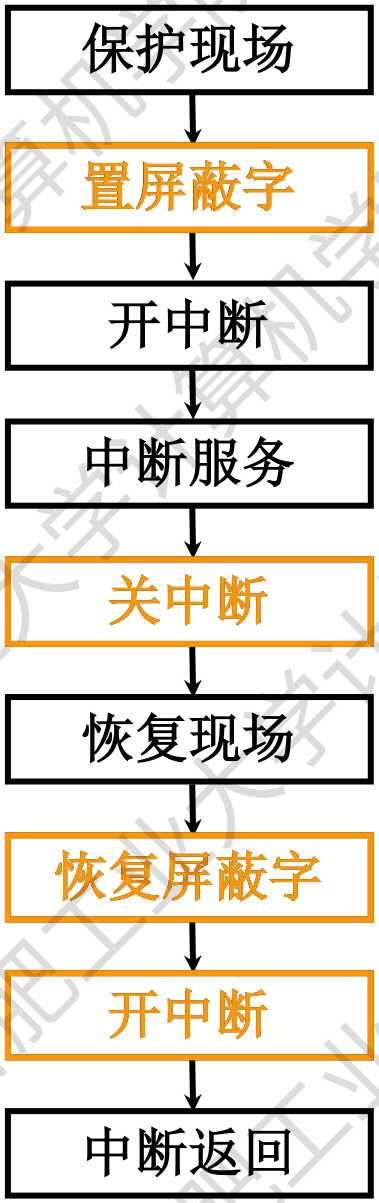
(4) 屏蔽技术的其他作用

可以 人为地屏蔽 某个中断源的请求

便于程序控制

(5) 新屏蔽字的设置

8.4



合肥工业大学计算机学院贾伟

4. 多重中断的断点保护

多重中断时，每次中断出现的断点都必须保存起来。中断系统对断点的保存都是在中断周期内由中断隐指令实现的，对用户是透明的。

断点可以保存在堆栈中，由于堆栈先进后出的特点，依次将程序的断点压入堆栈中。出栈时，按相反顺序便可准确返回到程序间断处。

断点也可保存在特定的存储单元内，例如约定一律将程序断点存至主存的0号地址单元内。由于保存断点是由中断隐指令自动完成的，因此三次中断的断点都将存入0地址单元，这势必造成前两次存入的断点k+1和L+1被冲掉。为此，在中断服务程序中的开中断指令之前，必须先将0地址单元的内容转存至别的地址单元中，才能真正保存每一个断点。

4. 多重中断的断点保护

8.4

(1) 断点进栈　　中断隐指令 完成

(2) 断点存入“0”地址　　中断隐指令 完成

中断周期　　0 ⟶ MAR

命令存储器写

PC ⟶ MDR　　断点 ⟶ MDR

(MDR) ⟶ 存入存储器

三次中断，三个断点都存入“0”地址

？ 如何保证断点不丢失？

例 题

1. 某机有五个中断源，按中断响应的优先顺序由高到低为L0,L1,L2,L3,L4，现要求优先顺序改为L4,L2,L3,L0,L1，写出各中断源的屏蔽字。

中断源	屏蔽字 0 1 2 3 4
L0 L1 L2 L3 L4	

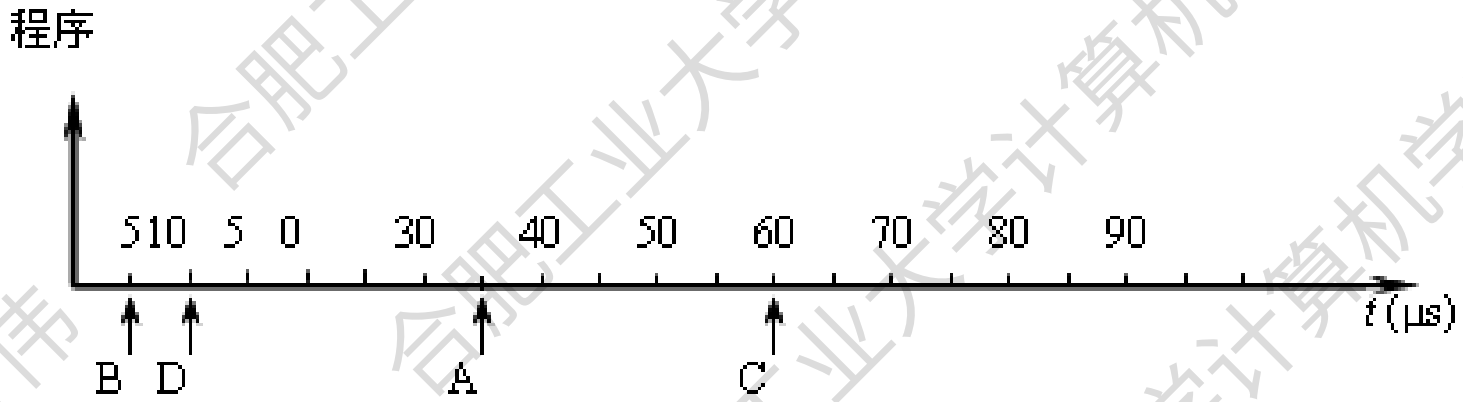
中断源	屏蔽字 0 1 2 3 4
L0	1 1 0 0 0
L1	0 1 0 0 0
L2	1 1 1 1 0
L3	1 1 0 1 0
L4	1 1 1 1 1

例 题

2. 设某机有四个中断源A、B、C、D，其硬件排队优先次序为A > B > C > D，现要求将中断处理次序改为D > A > C > B。

（1）写出每个中断源对应的屏蔽字。

（2）按下图时间轴给出的四个中断源的请求时刻，画出CPU执行程序的轨迹。设每个中断源的中断服务程序时间均为20μs。



答：（1）在中断处理次序改为D > A > C > B后，每个中断源新的屏蔽字如表所示。

中断源	屏蔽字			
	A	B	C	D
A	1	1	1	0
B	0	1	0	0
C	0	1	1	0
D	1	1	1	1

（2）根据新的处理次序，CPU执行程序的轨迹如图所示

